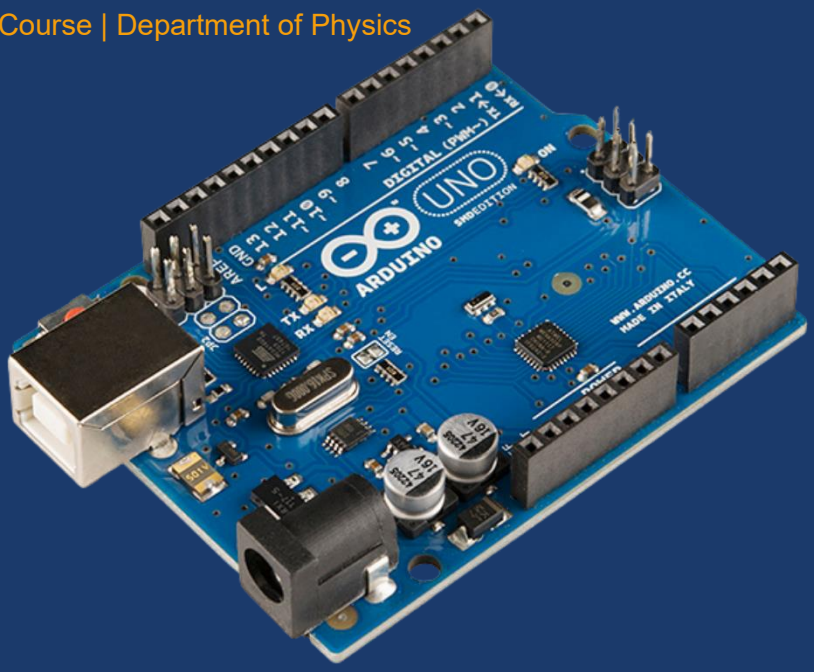
DMCK/PHY/AD__

Introduction to Smart Electronics with Arduino

Student Handbook

A beginner-friendly guide to
Arduino, Embedded Systems, Sensors, Actuators,Programming, and the Internet of Things

Add-On Course | Department of Physics



Deva Matha College, Kuravilangad

2026–27

How to Use This Handbook

This handbook is your companion before, during, and after the add-on course. It follows the exact order of the course syllabus.

Each module contains:

- **Plain-language explanations** of every concept, followed by a slightly deeper technical view
- **Real-world examples and analogies** to anchor abstract ideas
- **Practical Connection** boxes linking theory to the components on your bench
- **Code with line-by-line explanations** wherever programming appears
- **Figures** for understanding circuits, and components
- **Key Takeaways**, **Quick Review Questions**, and **Mini Activities** at the end

Enrichment sections — **Beyond the Workshop** and **Further Exploration** — are optional on a first read.

Icon legend:

- ⚙ **Practical Connection** — links reading to bench session
- ✓ **Key Takeaways** — essential points from each module
- **? Quick Review Questions** — check understanding
- ▬ **Mini Activities** — hands-on challenges
- ⚠ **Important** — safety and warnings
- ✗ **Common Mistake** — pitfalls to avoid
- **Tip** — helpful hints
- ➤ **Beyond the Workshop** — enrichment for the curious

Preparation Checklist

Task	How to Verify	Done?
Download and install Arduino IDE 2.x from arduino.cc/en/software	IDE opens without error	☐
Connect Arduino Uno with a USB data cable (not charge-only)	Power LED lights immediately	☐
Select: Tools → Board → Arduino AVR Boards → Arduino Uno	Status bar shows "Arduino Uno"	☐
Select: Tools → Port → (your port, e.g. COM3)	Port name visible in menu	☐
Upload Blink: File → Examples → 01.Basics → Blink → Upload	"Done uploading"; onboard LED blinks	☐

> ⚠ **IMPORTANT**
>
> A "charge-only" USB cable has only two wires inside (power and ground). It will charge your phone but cannot transfer data. If your board powers up but no ports appear in the IDE, swap the cable first.

Contents

Recommended Books

Online Courses

Open Source Project Ideas

Kit Bill of Materials — Component Identification Guide

Your kit contains the following components.

#	Component	Qty	Identification
1	Arduino Uno R3 board	1	Blue/black PCB with USB port, microcontroller chip, rows of pin holes
2	USB Type-B data cable	1	Square connector one end, USB-A other end (not a phone charging cable)
3	Breadboard	1	White plastic with holes in a grid, coloured rails on sides
4	Jumper wires	50	Flexible wires; usually multi-coloured set
5	LED	3	Clear/coloured plastic bead with 2 legs (one longer)
6	Resistors — 220 Ω	5	Small cylinder with bands: Red-Red-Brown-Gold
7	Resistors — 10 kΩ	3	Bands: Brown-Black-Orange-Gold
8	Push button (tactile switch)	1	Small square body with 4 legs; clicks when pressed
9	Potentiometer (10 kΩ)	1	Blue/black cylinder with 3 pins and rotating knob
10	LDR Module	1	Small board with photoresistor and 4 pins
11	Buzzer (active)	1	Small Board with Black cylinder and 2 pins
12	SG90 Servo motor	1	Small motor with 3-wire cable (orange, red, brown)
13	HC-SR04 Ultrasonic sensor	1	Small board with two silver cylinders and 4 pins

> ⚠ **IMPORTANT**
>
> ACTIVE vs PASSIVE BUZZER: This kit uses an ACTIVE buzzer. Apply 5V to make it beep — no tone() required. To identify: connect to 5V/GND briefly. If it beeps immediately, it is active. If silent, it is passive (needs tone()).

M1

Module 1

Introduction to Arduino and IoT

⏱ 3 Hours

Set the foundation: understand what a smart system is, how Arduino fits in, and identify every component in your kit.

1.1 What Is a Smart System?

Look around and you will find devices that make decisions: a washing machine that runs a cycle autonomously, a door that opens as you approach, an air conditioner that switches off when the room reaches a set temperature. These contain a tiny computer, some sensors, and a set of instructions.

A **smart system** is any everyday object that can *sense* its surroundings, *decide* what to do, and *act* — without continuous human control.

Type	What It Contains	Key Feature	Example
Regular system	Mechanical/electrical parts only	No computation; same every time	Old wall switch
Embedded system	Microcontroller + sensors	Local intelligence; standalone; no internet	Microwave, digital watch
IoT system	Microcontroller + sensors + internet	Connected to cloud; remotely monitored	Smart thermostat, fitness tracker, connected CCTV

Embedded system is the central idea of this workshop. The word *embedded* means the computer is hidden inside the object. A microwave has a computer inside but looks and works like a microwave — the complexity is buried.

An **IoT system** is an embedded system connected to a network. A standalone temperature sensor is embedded; the same sensor that uploads readings to a web dashboard you check from anywhere is IoT.

Automation Around Us

- **Traffic lights** cycle through colours on a timed and sensor-driven schedule.
- **Elevators** sense floor requests, compute the most efficient route, and move passengers autonomously.
- **Automatic doors** detect approaching people using sensors.
- **Automatic street lamps** detect darkness using a light sensor and switch on at dusk.
- **Car parking sensors** measure distance to obstacles and alert the driver with increasing urgency.

Activity 1.1 — Smart System Audit

Before the session: list ten devices you use regularly. For each, classify as Regular / Embedded / IoT and identify what it senses. Sample entry: "Ceiling fan with remote — Embedded — senses remote signal; decides speed; rotates faster or slower."

1.2 The Input → Processing → Output Model

This is the single most important idea in the entire workshop. Every project — from a blinking LED to a smart dustbin — is an instance of this three-stage pipeline.

- **Input** — Sensors detect something about the physical world and convert it into an electrical signal.
- **Processing** — The microcontroller runs your code, examines the input, and decides what should happen.
- **Output** — Actuators carry out the decision: lighting an LED, sounding a buzzer, rotating a motor.

Worked Examples of the Loop

Example 1 — Button → LED ON: Press a button → Arduino reads HIGH signal (input) → checks "Is button pressed?" (processing) → if yes, turns LED on (output).

Example 2 — Darkness → Light ON: LDR reads low light (input) → Arduino checks if below threshold (processing) → switches lamp on (output).

Example 3 — Object Near → Alarm: Ultrasonic sensor measures distance (input) → Arduino sees object too close (processing) → triggers buzzer (output).

Mini-Loop Worksheet — Activity 1.2

Device	Input (What it senses)	Decision Rule (Processing)	Output (What it does)
Automatic street lamp	LDR reads light level	If light < threshold → it is dark	Switch lamp ON
[Your example 1]			
[Your example 2]			

The Three Building Blocks

- **Input devices (sensors):** push button, potentiometer, LDR, ultrasonic sensor.
- **Processing unit (the brain):** the microcontroller. Reads inputs, makes decisions, controls outputs.
- **Output devices (actuators):** LEDs (light), buzzers (sound), servo motors (movement).

1.3 Meet the Arduino

Plain-Language View

Arduino is a small, beginner-friendly electronics board programmable from your computer. It reads signals from sensors and controls outputs like lights and motors. Inexpensive, easy to learn, and very forgiving of mistakes.

Deeper Technical View

An **Arduino board** is an open-source platform built around a **microcontroller** — a complete, tiny computer on a single chip. On the **Arduino Uno**, the main microcontroller is the **ATmega328P**.

"Open-source" means the board's circuit diagrams are published freely — anyone can study, modify, or manufacture their own version.

The Arduino Family — Choosing a Board

Board	Key Feature	Digital Pins	RAM	Best For
Arduino Uno R3	Standard starter board	14 (6 PWM)	2 KB	Learning, prototyping all sensor/actuator combinations
Arduino Nano	Compact, breadboard-friendly; same chip as Uno	14 (6 PWM)	2 KB	Projects where space is limited
Arduino Mega 2560	Many more pins; larger memory	54 (15 PWM)	8 KB	Complex projects needing many components or more code
ESP32 (Arduino-compatible)	Built-in Wi-Fi + Bluetooth; faster	34	520 KB	IoT and wireless projects; the natural "next step"

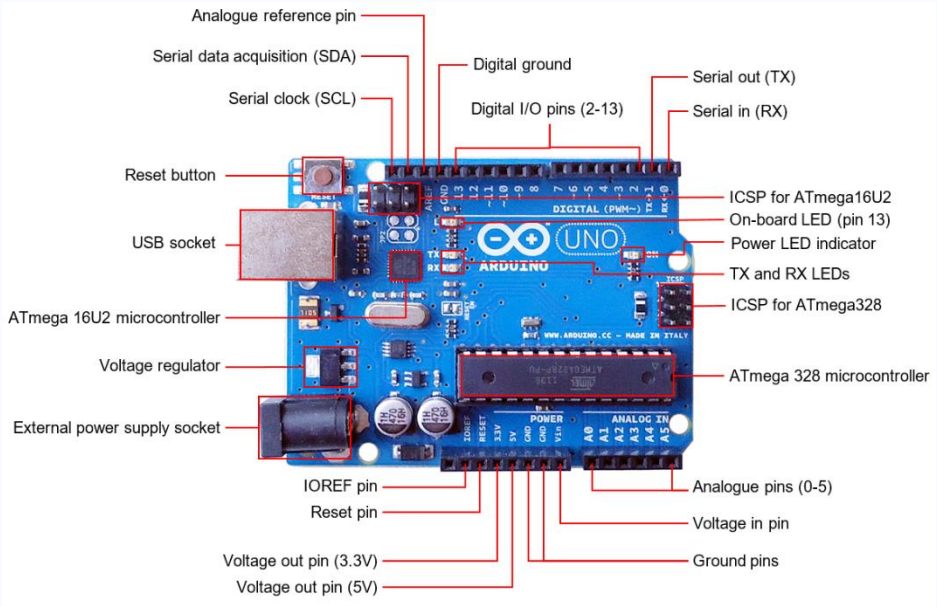
This add-on course uses the **Arduino Uno** because it is the most widely documented, most forgiving, and best supported board for first-time learners.

> ➤ **BEYOND THE WORKSHOP**
>
> The Uno carries two microcontroller chips. The main one (ATmega328P) runs your program. A second, smaller chip (ATmega16U2) handles USB communication between your computer and the main chip — you never program it directly.

1.4 Tour of the Arduino Uno

Before wiring anything, recognise the main parts of the board. Study Figure 1.1, then locate each part on your physical board.

Figure 1.1: Labelled diagram of an Arduino Uno



Part	Location on Board	What It Does
USB Port	Left edge (square connector)	Connects to computer for code upload and 5V power supply
Power Jack	Left edge (round)	Accepts 7–12V DC for standalone (no computer) operation
ATmega328P	Large black rectangle, centre	The main microcontroller — runs your program
Digital Pins 0–13	Top edge, right side	Read or write HIGH/LOW; each can source/sink up to 40mA
PWM Pins (3,5,6,9,10,11)	Top edge — marked with ~	Support analogWrite() for variable outputs
Analog Pins A0–A5	Bottom-right edge	Read variable voltage 0–5V; inbuilt ADC converts it to 0–1023
5V Pin	Bottom edge	Provides regulated 5V to power sensors and components
GND Pins (×3)	Various positions	Common ground reference for all circuits
Reset Button	Left side	Restarts program from beginning
Pin 13 LED	Near ATmega chip	Onboard LED connected to digital pin 13

Digital pins deal in two states: HIGH (5V, "on") or LOW (0V, "off").

Analog pins read continuous voltage between 0V and 5V. The pins are **holes (called headers)** — push a jumper wire in to connect directly to the microcontroller.

Activity 1.3 — Board Component Hunt

Using Figure 1.1, locate each part on your physical Arduino Uno: USB port, power jack, ATmega328P, digital pins, analog pins, PWM (~) pins, GND pins, 5V pin, reset button, TX/RX LEDs, Pin 13 LED.

1.5 Components You Will Use

Component	Senses / Produces	Type
Push button	A press — on/off	Input (sensor)
Potentiometer	Rotation of knob (variable)	Input (sensor)
LDR (photoresistor)	Intensity of Light	Input (sensor)
HC-SR04 ultrasonic sensor	Distance to object (2–400cm)	Input (sensor)
LED	Light (on/off or variable brightness)	Output (actuator)
Buzzer (active)	Sound (beep)	Output (actuator)
SG90 servo motor	Precise angular movement (0°–180°)	Output (actuator)

1.6 Workshop Safety

Do

- Handle components gently by their bodies, not by bending legs repeatedly.
- Keep liquids away from the boards.
- Disconnect power (unplug USB) before changing wiring.
- Double-check every connection against your wiring plan before powering on.
- Ask for help whenever unsure — no question is too basic.

Don't

- Touch a powered circuit while rearranging it.
- Rush connections — a single misplaced wire can destroy a component.
- Leave component legs exposed where they can touch and short.

> ⚠ **IMPORTANT**
>
> **LED without a resistor:** Without a 220Ω resistor, since an LED has low resistance, will draw excessive current, and burns out permanently, as well as damage the Arduino.
>
> **Reversed polarity:** Connecting a polarised component (LED, buzzer) backwards can destroy it instantly. Check polarity before powering on.
>
> **Short circuit (5V touching GND directly):** Can draw excess current and damage the voltage regulator. Unplug USB immediately if you suspect a short.

1.7 IoT in Society: Benefits, Risks, and Responsibility

Connected devices have transformed daily life. As an engineer or maker, your designs will affect real people.

What IoT Makes Possible

- **Independent living support:** Smart sensors can detect unusual patterns and alert a carer, enabling elderly people to live more independently.
- **Industrial efficiency:** Factory sensors monitor machinery in real time, predicting failures before they happen.
- **Environmental monitoring:** Remote sensors in forests, rivers, and oceans collect data impossible to gather manually.

Real Tradeoffs

- **Privacy:** Connected devices collect data about your habits, location, and health. Who owns that data? Who can access it?
- **Security:** Devices with weak default passwords become vulnerabilities. A 2016 attack used 100,000 compromised IoT devices to overwhelm major websites.
- **Dependence:** When a network fails, or a manufacturer shuts down their server, devices you rely on may stop working.

Reflection Questions

1. A smart dustbin opens when your hand approaches. What data could this sensor collect?

2. A fitness wristband tracks heart rate, sleep, and location 24 hours a day. List three benefits and two privacy concerns.
3. A village installs soil moisture sensors connected to the internet to automate irrigation. What happens if the internet connection is cut for a week?

✓ KEY TAKEAWAYS

A **smart system** senses, decides, and acts. The three families: regular (no computation), embedded (local intelligence, standalone), IoT (network connected).

The **Input → Processing → Output loop** underlies every project in this workshop.

The **Arduino Uno** uses the ATmega328P microcontroller. Digital pins handle on/off; analog pins read variable values.

Sensors are inputs; actuators (LEDs, buzzers, servos) are outputs.

IoT brings genuine benefits and genuine risks. Responsible design considers privacy, security, and dependence.

? QUICK REVIEW QUESTIONS

- In your own words, what distinguishes an embedded system from an IoT system? Give one example of each.
- Describe an automatic street lamp using the Input → Processing → Output model. Name the specific sensor and actuator.
- Which Arduino pins would you use to read a light sensor? Why can you not use a digital pin?
- What is the name and role of the Arduino Uno's main microcontroller? What does the second chip do?
- Is the Arduino Uno (without any wireless module) an IoT device? Justify your answer.
- Why must you disconnect USB power before changing wiring?
- List three real-world privacy risks associated with IoT devices in a home.
- Give two examples of input devices and two examples of output devices from your kit.
- A pacemaker has a processor and controls a vital body function but is not internet-connected. Is it embedded, IoT, or both? Explain.

✒ MINI ACTIVITIES

Spot the Loop: Pick three appliances in your home. For each, write the specific input, the decision rule, and the output. Sketch each as: sensor → microcontroller → actuator.

Classify It: For five devices you own, label each as Regular / Embedded / IoT and justify your choice.

M2

Module 2

Arduino IDE Installation and Basic Electronics

⏱ 5 Hours

Install the software, connect your board, master the breadboard, and run your first program. By the end you will have made an LED blink — the "Hello World" of hardware.

2.1 The Arduino Ecosystem

An Arduino project always involves three cooperating parts that depend on each other:

- **The hardware** — the Arduino board and any components wired to it.
- **The software (the IDE)** — where you write, verify, and upload your code.
- **The code (your sketch)** — the instructions that bring the hardware to life.

The Arduino term for a program is a **sketch**. A shield is an add-on board that stacks on top of the Arduino, extending its capabilities (e.g. a Wi-Fi shield, an SD card shield). Shields come with **libraries** — pre-written code that hides complex details so you can use advanced hardware with simple commands. You will not need shields in this course, but understanding the word "library" matters — you will use one in Module 5.

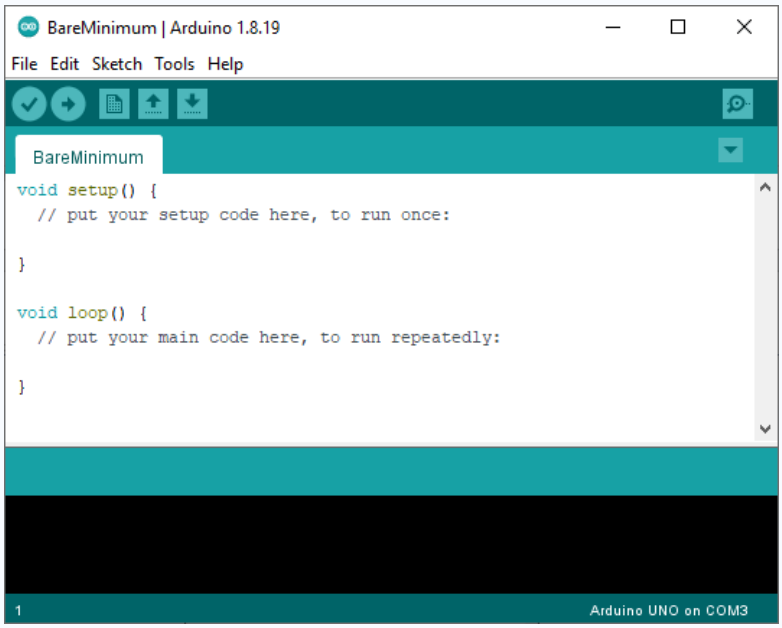
2.2 What Is the Arduino IDE?

IDE stands for **Integrated Development Environment**. It is the single program that bundles everything you need: write code, check for errors, and send it to the board. Think of it as your coding command centre.

The IDE bundles several tools that would otherwise be separate applications:

- A **text editor** to type your sketch
- A **compiler** that translates your readable C code into machine code the chip understands
- An **uploader** that copies the finished program to the board through the USB cable
- A **Serial Monitor** for viewing text messages sent from the board to your computer

Figure 2.1: The Arduino IDE Interface



Button / Control	What It Does	When to Use
Verify (✓)	Compiles code and checks for errors — does NOT upload	Check code for errors before you have a board connected
Upload (→)	Compiles AND sends code to the connected board	When you are ready to test on real hardware
Serial Monitor	Opens the live text window between board and computer	Debugging; watching sensor values; checking program flow
Board selector	Shows/changes the target board type	Must be set to "Arduino Uno" for this workshop
Port selector	Shows/changes the USB port the board is on	Must match the port your board is connected to

2.3 Installing the IDE

The process is straightforward on Windows, macOS, and Linux. Follow these steps exactly, in order:

1. Go to **arduino.cc/en/software** and download the **Arduino IDE 2.x** installer for your operating system.
2. Run the installer. On Windows, a dialog will ask permission to install **USB drivers** — click **Yes**.
3. Launch the IDE. A blank sketch appears with empty `setup()` and `loop()` functions — this is normal.

> **TIP**
>
> A **driver** is a small piece of software that lets your operating system communicate with a specific hardware device. The Arduino USB driver tells Windows how to talk to the board through the USB cable. macOS and Linux usually include the necessary drivers automatically — no extra steps needed.

2.4 Connecting the Board — Step by Step

1. Plug the Arduino into your computer with a **USB data cable**. The board's small green **power LED** lights immediately.
2. In the IDE, from Tools, click the board selector. A dropdown appears. Choose **Arduino Uno**. The status bar now shows "Arduino Uno."
3. Click the port selector. On Windows it shows as COM3, COM4, etc. On macOS/Linux it shows as /dev/cu.usbmodem... or /dev/ttyACM0. Select the port that appeared when you connected the board.
4. If only one port appears, select it. If no ports appear at all, work through the troubleshooting table at the end of this module before continuing.

What is a port? A port is the operating system's name for a communication channel. Each device connected to your computer through USB gets its own numbered port. Selecting the correct port tells the IDE which USB channel leads to your Arduino.

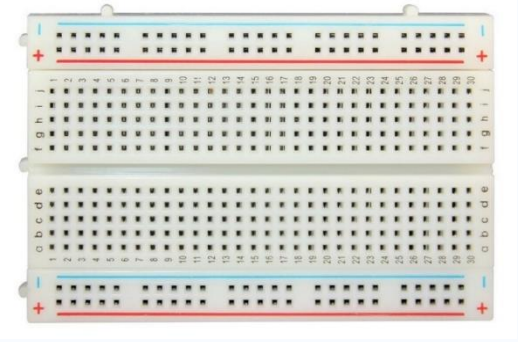
> **✗ COMMON MISTAKE**
>
> Uploading to the wrong board setting (e.g., Arduino Mega when you have an Uno) does not damage anything, but the upload will fail with an obscure error. The most common new-user error is forgetting to change the board setting after opening the IDE for the first time.
>
> Selecting the wrong COM port also causes no damage — the IDE simply reports a "port not found" error. If you are unsure which port is correct, try each one in turn.

2.5 Breadboards and Jumper Wires

The Breadboard

A **breadboard** is a reusable board with hundreds of holes that lets you build circuits without soldering. You simply push component legs and wires into the holes; spring-loaded metal clips inside the board grip them and make the electrical connections. Because nothing is permanent, you can rearrange freely — perfect for learning and prototyping.

Figure 2.2: Breadboard



The crucial fact about breadboard connections:

- **The main grid (rows):** Holes are connected in short horizontal rows of five. Any two wires placed in the same row of five are electrically joined. A channel down the middle divides the top half from the bottom half — holes across the channel are NOT connected.
- **The power rails (columns):** The long columns marked + and − on each side run vertically and are connected along their whole length. Run one wire from the Arduino's 5V pin to the + rail, and every hole on that rail is at 5V.

Best practices for breadboard wiring:

- Use **short wires** — a shorter wire is less likely to be accidentally displaced.
- **Colour code** your wires: Red = 5V power; Black or Blue = GND; other colours = signals. This convention makes troubleshooting much faster.
- Insert component legs **straight** — bending legs sideways weakens them and makes them hard to reuse.
- Pull wires out **gently and straight** — sideways force bends the pins.

Jumper Wires

Jumper wires have metal pins at their ends. Three end types exist: **male–male** (pin both ends, for connecting breadboard points), **male–female** (pin one end, for connecting an Arduino pin to a sensor module), and **female–female** (socket both ends, for connecting two modules). This workshop uses mostly male–male.

2.6 Component Spotlight: The LED

An **LED (Light Emitting Diode)** is a semiconductor component that emits light when current flows through it in the correct direction. Unlike a regular light bulb, an LED is *polarised* — it has a positive and a negative terminal.

- **Long leg = Anode (+).** Connects toward positive voltage (5V or a resistor connected to 5V).
- **Short leg = Cathode (−).** Connects toward ground (GND).
- **Flat edge:** A small flat on the plastic base also marks the cathode (−) side — useful if the legs have been trimmed to equal length.

> ⚠ **IMPORTANT**
>
> Always place a **resistor (220 Ω)** in series with every LED. Without it, too much current flows, and the LED will flash very brightly once and burn out permanently — sometimes in under a second. A blown LED cannot be repaired. The resistor protects it by limiting current to a safe level.

A Quick Electricity Primer: Why 220 Ω?

You will wire resistors in this module without needing to calculate everything from scratch, but understanding the reasoning makes you a better debugger. Three quantities define a simple electrical circuit:

- **Voltage (V)** — Your Arduino pin outputs 5V.
- **Current (I)** — Measured in milliamps (mA). A red LED can safely carry about 20mA.
- **Resistance (R)** — Measured in ohms (Ω). A resistor limits current.

The relationship between the three is **Ohm's Law: V = I × R**

Why 220 Ω for an LED? An LED "drops" about 2V across itself (this is its forward voltage). The rest of the supply voltage (5V − 2V = 3V) must be dropped across the resistor. If we want to limit current to a safe 15mA:

R = V ÷ I = 3V ÷ 0.015A = 200 Ω

The nearest standard resistor value above 200 Ω is **220 Ω**, which gives about 13.6mA — comfortably safe. Using 330 Ω gives about 9mA, which is dimmer but equally safe.

> **TIP**
>
> You do not need to calculate resistor values from scratch during the workshop. The rule is simple: every LED gets a 220 Ω resistor in series. Always. No exceptions. The calculation above simply explains why 220 Ω is correct.

2.7 Your First Program: Blink

Now the satisfying part. The Blink example makes the LED built into the Arduino board flash on and off. It is the "Hello World" of hardware — proof that your entire toolchain (IDE, drivers, board, USB) is working.

Loading and Running It

1. Click **File → Examples → 01.Basics → Blink** in the IDE.
2. A new sketch opens with the Blink code already written.
3. Click **Upload** (the right-arrow button).
4. The IDE compiles the code (a few seconds), then uploads it. The TX/RX LEDs near the USB port blink rapidly during transfer.
5. After "Done uploading" appears, the onboard LED (next to pin 13) begins blinking once per second.

If that happens: your entire toolchain is working and verified. You are ready for every session that follows.

The Blink Sketch

```
void setup() {
  // runs ONCE when the Arduino powers on or resets
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  // runs FOREVER, repeating top to bottom endlessly
  digitalWrite(LED_BUILTIN, HIGH);  // send 5V to pin — LED ON
  delay(1000);                      // pause 1000 milliseconds (1 second)
  digitalWrite(LED_BUILTIN, LOW);   // send 0V to pin — LED OFF
  delay(1000);                      // pause again
}
```

Line-by-Line Explanation

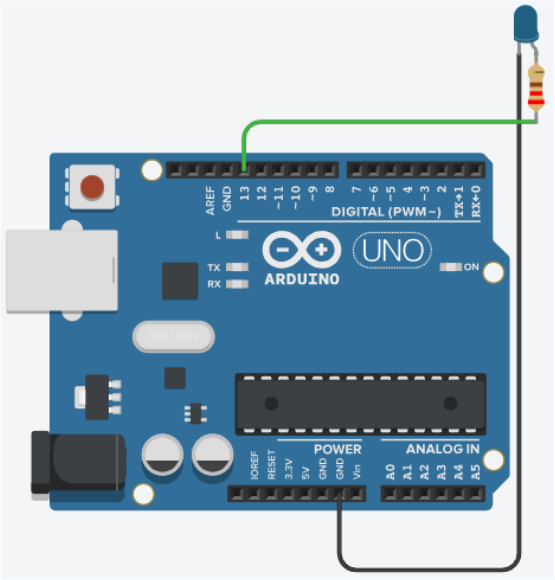
- `void setup()` — runs exactly **once** when the board powers on or when the reset button is pressed. Used for preparation: setting pin modes, starting communication.
- `pinMode(LED_BUILTIN, OUTPUT)` — tells the Arduino that this pin will send signals (it is an output). `LED_BUILTIN` is a built-in name that the IDE defines as pin 13 on the Uno.
- `void loop()` — runs **forever** after setup() finishes, repeating from top to bottom for as long as the board has power.
- `digitalWrite(LED_BUILTIN, HIGH)` — sends 5V to the pin, turning the LED on.
- `delay(1000)` — pauses the program for 1000 milliseconds. While paused, nothing else happens.
- `digitalWrite(LED_BUILTIN, LOW)` — sends 0V, turning the LED off.

Modifications to Try

Change the speed. Replace both `1000`s with `200` to blink fast, or `2000` to blink slowly.

Use an external LED (the most important modification):

Figure 2.3



External LED on Pin 7

```
const int ledPin = 7;    // assigns the pin to a variable

void setup() {
  pinMode(ledPin, OUTPUT);  // configure ledPin as output
}

void loop() {
  digitalWrite(ledPin, HIGH);  // LED ON
  delay(500);
  digitalWrite(ledPin, LOW);   // LED OFF
  delay(500);
}
```

Using `const int ledPin = 7;` creates a **named constant**. If you later move the LED to pin 8, you change one line at the top instead of searching through the whole sketch. This habit becomes essential in longer programs. The keyword `const` tells the compiler this value cannot be changed during the program — it is a configuration value, not a working variable.

What Happens During Upload?

Your code travels from the computer through the USB cable directly into the Arduino's flash memory — ready to run instantly.
Flash memory is **non-volatile** — it keeps its contents even when power is removed. That is why your Arduino remembers its program: unplug it, plug it back in, and Blink restarts automatically. The variable values in SRAM, however, reset every time power is removed — they start from their initial values each boot.

2.8 Component Spotlight: The Buzzer

A **buzzer** produces sound when powered. Your kit contains an **active buzzer** — it has a small oscillator circuit inside and requires only a digital HIGH signal to produce a standard tone. This distinguishes it from a passive buzzer, which requires the `tone()` function to drive it.

Wiring the buzzer:

- The buzzer has a + marking on the top. Connect the + leg to any digital pin.
- Connect the other leg to GND.
- In code: `digitalWrite(buzzerPin, HIGH)` to beep; `digitalWrite(buzzerPin, LOW)` to stop.

Using tone() with the buzzer (advanced): The `tone(pin, frequency, duration)` function drives any digital pin at a specific frequency. Parameters: pin number, frequency in Hz (220–4000 Hz gives clear tones), optional duration in milliseconds. Call `noTone(pin)` to silence it. Use `tone()` when you need a specific pitch for musical notes or varying alarm tones.

Active Buzzer Beep

```
// Active buzzer — simple beep
int buzzerPin = 8;

void setup() {
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  digitalWrite(buzzerPin, HIGH);   // buzzer ON
  delay(500);
  digitalWrite(buzzerPin, LOW);    // buzzer OFF
  delay(500);
}
```

Pitch-Controlled Buzzer with tone()

```
// tone() function — specific pitch (passive buzzer)
int buzzerPin = 8;

void loop() {
  tone(buzzerPin, 1000, 500);  // 1000 Hz for 500 ms
  delay(600);                  // wait before next tone
}
```

2.9 Tinkercad Circuits: Simulate Before You Build

Before touching a single wire, you can build and test your entire Arduino circuit on a computer. Tinkercad Circuits is a free, browser-based electronics simulator used worldwide by students, teachers, and makers. Think of it as a virtual lab bench — every component in your kit is available, the wiring behaves like real wiring, and the Arduino runs your actual code.

What Tinkercad Circuits Is

Tinkercad Circuits (tinkercad.com) lets you drag and drop components onto a virtual breadboard, connect them with virtual wires, write and run Arduino code, and see the results in real time — all without any physical hardware. The simulation engine models real electrical behaviour: an LED without a resistor will burn out just as it would in real life, and a floating input pin will read randomly, just as it does on the bench.

Why Simulation Is Useful

- **Safe experimentation:** You can short-circuit, wire incorrectly, and reverse polarity in the simulator with zero risk. No components burn out, no frustration.
- **Test before building:** Debug your code logic and circuit layout before touching physical components. Catch pin number mistakes and logic errors early.
- **Instant replay:** Pause, rewind, and step through simulations to understand exactly what is happening at each moment.
- **Available anywhere:** A browser and an internet connection are all you need. No hardware required.

2.10 Basic Troubleshooting

Problem	Likely Cause	Fix
Upload fails — "port not found"	Wrong port selected or board not connected	Check Tools → Port. Try a different USB port on your computer
Upload fails — "programmer not responding"	Wrong board type selected	Check Tools → Board → Arduino Uno
Board not detected (no port appears)	Charge-only cable, or missing USB driver	Try a different cable. On Windows, reinstall USB drivers from Device Manager
LED does not light	Reversed LED polarity (anode and cathode swapped)	Flip the LED in the breadboard so the long leg faces the resistor side
LED very dim	Very high resistor value	Add a 220Ω resistor in series with the LED
LED burned out (no light and not fixable)	LED connected directly to 5V without resistor	Replace the LED; always wire resistor first
Compile error with red message	Missing semicolon, bracket, or typo	Read the error message — the line number points to or near the problem

Garbled Serial text	Baud rate mismatch	Match Serial.begin(9600) in code to the baud rate dropdown in the monitor (9600)

TIP

When in doubt — unplug, double-check every connection against your wiring diagram, re-upload. The most common mistake is a wire in the wrong row of the breadboard, one row off from where it should be.

TIP

Compiler error messages: The IDE highlights the line the compiler flags with a red underline or pink background. However, the actual error is *often on the line before the highlighted one* — a missing semicolon on line 10 causes an error that the compiler reports on line 11. Always check one line above the highlighted line first.

✓ KEY TAKEAWAYS

The Arduino workflow is **write** → **verify** → **upload** → **run**, all managed inside the IDE.

A sketch always has `setup()` (runs once at start) and `loop()` (runs forever).

A breadboard connects components without soldering. **Rows of five** are connected; **power rails** run the full length; the centre channel separates the two halves.

LEDs are polarised — **long leg to +**, short leg to −. Always add a **220Ω resistor in series**.

`digitalWrite(pin, HIGH/LOW)` turns a pin on or off; `delay(ms)` pauses the program.

Pull-up resistors (or `INPUT_PULLUP`) give floating input pins a defined default state — critical for reliable button circuits (Module 3).

Programs live in **flash memory** (non-volatile); variables live in SRAM (lost on power-off).

An **active buzzer** makes sound with just `digitalWrite(HIGH)`; a passive buzzer needs `tone()`.

? QUICK REVIEW QUESTIONS

- What three tools does the Arduino IDE bundle into one application?
- On a breadboard, which holes are electrically connected to each other? What separates the two halves?

- An LED needs 2V forward voltage and you want 15mA of current from a 5V pin. Calculate the minimum resistor value needed. Show your working using Ohm's Law.
- What is the difference between `setup()` and `loop()`? Give an analogy from daily life.
- What does `delay(500)` do? Name one situation where using `delay()` would cause a problem.
- Why does the Arduino keep its program after you unplug it? Which type of memory stores the program?
- Your upload fails with "port not found." List three things to check, in order.
- What is a floating pin? How does a pull-up resistor solve the problem?
- What is the difference between an active and a passive buzzer? How do you identify which type you have?
- What does `const int ledPin = 7;` do that `int ledPin = 7;` does not?

MINI ACTIVITIES

Blink Your Own LED: Wire an external LED to pin 8 through a 220Ω resistor (use Figure 2.5 as a guide). Modify Blink to control pin 8 instead of LED_BUILTIN. Upload and verify the LED blinks.

Two-Speed Blink: Make the LED stay ON for 1000ms but OFF for only 100ms. Describe the visual effect. Why does the LED appear to be "on most of the time"?

SOS Challenge: Morse code SOS is · · · — — — · · · (three short, three long, three short). Short = 200ms on, 200ms off. Long = 600ms on, 200ms off. Write the sketch, upload, and demonstrate to a partner.

Cable Test: Try uploading Blink with a known charge-only phone cable. Record what the IDE reports. Then switch to your data cable and repeat. Document the difference in behaviour.

M3

Module 3

Beginner Programming, LEDs, and Interactive Outputs

⏱ 7 Hours

Become a programmer: master the Arduino skeleton, control multiple outputs, read your first input, debug with the Serial Monitor, and build a reflex game.

3.0 Variables and Data Types: A Quick Primer

Before writing any program longer than Blink, you need to understand **variables** — one of the most fundamental ideas in all programming. A variable is a **named box in memory** that holds a value your program can read and change.

Declaring a Variable

In C (the language Arduino uses), you must declare every variable before using it. A declaration has three parts:

```
datatype  name  =  initial_value;
```

For example: `int buttonPin = 2;` — declares an integer variable named `buttonPin` with a starting value of 2.

Common Data Types in Arduino Programs

Type	Holds	Range (on Arduino Uno)	When to Use
int	Whole numbers	−32,768 to 32,767 (16-bit)	Pin numbers, sensor values, loop counters — the default choice
long	Large whole numbers	−2,147,483,648 to 2,147,483,647 (32-bit)	Timing values (microseconds), distance calculations that exceed int range
boolean	True or false only	true / false (or 1 / 0)	Flags, button states, on/off conditions
float	Decimal numbers	~6–7 significant digits	Voltage calculations, temperatures — use sparingly (slow on Uno)
const int	A whole number that never changes	Same as int	Pin numbers and thresholds — prevents accidental change

The most important rule: A variable declared inside a function (inside `{ }`) only exists inside that block. A variable declared at the top of the sketch (before `setup()`) is available everywhere. **For button states, LED states, and all variables you want to persist across loop() calls, declare them at the top of the sketch.**

Variable Scope — Where Variables Live

```
// Variables declared at the top — available everywhere
const int ledPin = 9;       // pin number — const because it never changes
const int buttonPin = 2;    // pin number
int buttonState = 0;        // holds the current button reading — changes every
loop
boolean ledOn = false;      // tracks whether LED is currently on or off

void setup() {
  // Variables declared here only exist inside setup()
  int tempVar = 5;  // only accessible within setup()
}

void loop() {
  // ledPin, buttonPin, buttonState, ledOn are all accessible here
  // tempVar is NOT accessible here — it was declared inside setup()
}
```

See Appendix A, Sections A.3–A.4 for a complete explanation of variable types, arithmetic operators, and comparison operators.

3.1 Anatomy of an Arduino Program

Every Arduino sketch has exactly two essential functions. No exceptions.

The Arduino Skeleton

```
void setup() {
  // runs ONCE when the Arduino starts (power-on or reset button)
}

void loop() {
  // runs FOREVER, repeating endlessly for as long as board has power
}
```

setup() is like waking up and preparing for the day. It runs a single time. Use it for:

- Setting pin modes (`pinMode`)
- Starting the Serial connection (`Serial.begin()`)
- Initialising sensors, displays, or libraries

loop() is your daily routine that never stops. After setup() finishes, the Arduino jumps immediately to loop() and runs it continuously — top to bottom, then back to the top, forever, like breathing.

setup() — happens once	loop() — repeats forever
Wake up in the morning	Breathe
Brush your teeth	Heartbeat
Get dressed	Blink your eyes
Configure your tools	Do the actual work

Concept summary: setup() initialises; loop() does the ongoing work. This is the complete skeleton — every Arduino sketch you ever write follows this pattern.

3.2 Digital Output and digitalWrite()

A **digital signal** has exactly two states:

- **HIGH = ON = 1** (approximately 5V on the output pin)
- **LOW = OFF = 0** (0V on the output pin)

It is exactly like a wall switch — on or off, nothing in between. This binary thinking is how computers work at the lowest level.

Controlling Outputs with digitalWrite()

```
digitalWrite(pin, HIGH);   // send 5V to pin — device ON
digitalWrite(pin, LOW);    // send 0V to pin — device OFF
```

Before using a pin as an output, you must declare it in `setup()`:

```
pinMode(7, OUTPUT);   // pin 7 will SEND signals
```

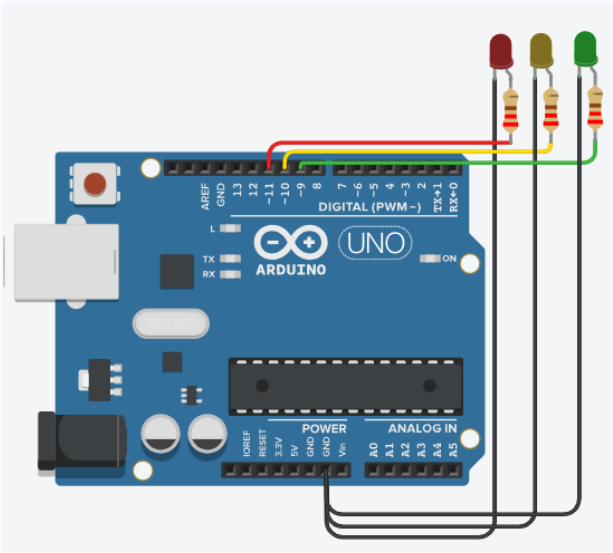
The Blink cycle is a four-step process that repeats endlessly: LED ON → Wait → LED OFF → Wait → (repeat). Each step is one instruction; the delay() controls speed. The same ON/OFF logic appears everywhere: phone charging LEDs, traffic signals, status indicators. You have been reading digital signals your whole life.

> ⚙ **PRACTICAL CONNECTION**
>
> Mastering `digitalWrite()` is the foundation for every output you will ever drive — from a single indicator LED to a servo's control signal. Everything uses the same HIGH/LOW concept you are learning now.

3.3 Multiple LEDs and Sequencing

The Arduino controls each pin independently, so you can drive several LEDs simultaneously. A classic example is a traffic light sequence:

Figure 3.4: Traffic Light Circuit Layout



Traffic Light Sequence

```
const int redPin    = 11;
const int yellowPin = 10;
const int greenPin  = 9;

void setup() {
  pinMode(redPin,    OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin,  OUTPUT);
}

void loop() {
  // GREEN phase — go
  digitalWrite(greenPin, HIGH);
  delay(3000);
  digitalWrite(greenPin, LOW);

  // YELLOW phase — prepare to stop
  digitalWrite(yellowPin, HIGH);
  delay(1000);
  digitalWrite(yellowPin, LOW);

  // RED phase — stop
  digitalWrite(redPin, HIGH);
  delay(3000);
  digitalWrite(redPin, LOW);
  // loop() returns to top — green phase starts again
}
```

Each LED needs its own digital pin, its own 220Ω resistor, and a connection to the GND rail.

LED Sequencing — Creating Movement

By turning a row of LEDs on and off in sequence, you can make light appear to "chase" or "wave" across the row — like the animated signs you see in shops. The timing of your delays controls how fast the pattern moves. This introduces the idea that the *order* of instructions creates the behaviour — not just what instructions you give, but *when* you give them.

3.3c Running LEDs and Chasing Effects

When you turn LEDs on and off in a specific order with short delays between each step, the light appears to move. This is the principle behind LED strip animations, chaser lights on signs, and running indicator lights in cars. The Arduino controls each LED independently, and the delay() values control the speed of the apparent motion.

Three classic patterns you can implement with five LEDs on pins 7–11:

- **Forward chaser:** LEDs turn on one at a time from left to right, then off from left to right. Creates a beam sweeping forward.
- **Bounce (Knight Rider):** LED moves forward then reverses direction, bouncing back and forth across the row.
- **Sequential wipe:** All LEDs turn on one by one (filling up), then all turn off one by one.

Timing creates the illusion of motion. At delay(50) the pattern moves so fast it appears as a blur. At delay(300) each individual step is clearly visible. Experiment with different delay values to find the speed that looks most fluid for your effect.

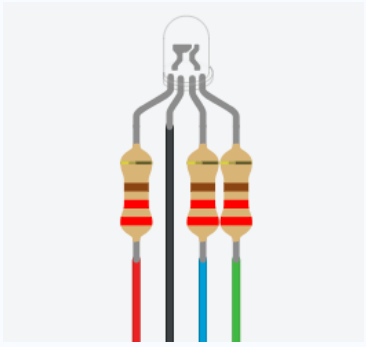
3.3b RGB LEDs: Mixing Light to Make Any Colour

So far you have used single-colour LEDs — they produce one fixed colour. An RGB LED contains three LEDs in one package: Red, Green, and Blue. By adjusting the brightness of each colour channel independently using PWM, you can mix them together to produce almost any colour — millions of combinations from a single component.

What an RGB LED Is

An RGB LED is a single component with four legs. Three of those legs connect to the three colour channels (Red, Green, Blue), and the fourth leg is a shared common connection. Lighting just the Red channel gives red light. Lighting Red and Green together gives yellow. Lighting all three equally gives white. Turning all three off gives no light at all.

Figure 3.8: RGB LED



Common Cathode vs Common Anode

RGB LEDs come in two configurations. The most common in beginner kits is common cathode: the longest leg connects to GND, and each colour pin is driven HIGH to light up that channel. Less common is common anode: the longest leg connects to 5V, and each colour pin is driven LOW to turn on that channel. This handbook uses common cathode wiring — check your kit's documentation if results seem inverted.

Wiring the RGB LED

- **R pin:** connect to a 220 Ω resistor, then to a PWM pin (e.g., pin 9)
- **G pin:** connect to a 220 Ω resistor, then to a PWM pin (e.g., pin 10)
- **B pin:** connect to a 220 Ω resistor, then to a PWM pin (e.g., pin 11)
- **Common (longest leg):** connect directly to GND

RGB LED can be used using a code similar to the one used for traffic lights. To vary intensity of each color, setColor() function can be used, which uses PWM.

setColour(r, g, b) is a custom helper function you define. Calling setColour(255, 0, 0) sends full brightness to Red, zero to Green and Blue — producing pure red light.

Real-World Applications of RGB LEDs

- Status indicators — smart devices use a single RGB LED to show different modes: solid blue = connected, blinking red = error, green = charging complete.
- Mood lighting — smart bulbs use arrays of RGB LEDs, mixing colours to create warm white, cool white, or any ambient colour requested.
- Gaming peripherals — keyboards, mice, and headsets use individually addressable RGB LEDs under microcontroller control.

- Display technology — a screen pixel is essentially one RGB LED element; your phone screen contains millions of them.

MINI ACTIVITY

RGB Colour Lab: modify setColour() calls to mix your own colours. Challenge: find the values that produce orange (hint: lots of Red, some Green, no Blue). Record your successful combinations in a table.

3.4 Inputs: Giving the Arduino Senses

Outputs alone make the Arduino speak. Inputs let it listen. With inputs, the Arduino can react to the world — just as you use your eyes and ears to make decisions.

Just like outputs, before using a pin as an input, you must declare it in `setup()`:

```
pinMode(7, INPUT);  // pin 7 will RECEIVE signals
```

Component Spotlight: The Push Button

A push button is a **temporary switch**. It closes a circuit only while pressed, letting current flow. Release it, and the connection breaks. A standard tactile push button has four legs connected internally in two pairs. Pressing the button bridges the two pairs and completes the circuit.

Without proper wiring, a push button connected to a digital input can produce unreliable readings. Understanding why this happens is the key to wiring buttons correctly.

Why Do We Need a Resistor?

A digital input pin can detect only two voltage levels:

- **HIGH** (approximately 5 V)
- **LOW** (approximately 0 V)

When a button is connected directly to an input pin, the pin is left **disconnected** whenever the button is not pressed. In this condition, the pin is said to be **floating**.

A floating pin has no defined voltage and can randomly read HIGH or LOW by picking up electrical noise from the surroundings. This causes unpredictable behaviour.

A resistor is therefore used to give the input pin a **default state** whenever the button is not pressed.

There are two common methods:

- Pull-Down Resistor
- Pull-Up Resistor

The Correct Way to Wire a Button

Method 1: External Pull-Down Resistor

In a pull-down circuit, the resistor connects the input pin to **GND**.

When the button is not pressed, the resistor keeps the pin at **LOW**.

When the button is pressed, the pin is connected directly to **5 V**, overriding the resistor and making the input **HIGH**.

Wiring

- One side of the button → **5 V**
- Other side → **Digital Pin**
- **10 kΩ resistor** between Digital Pin and **GND**

Button	Pin State
Released	LOW
Pressed	HIGH

```
const int buttonPin = 2;
const int ledPin = 13;

void setup() {
  pinMode(buttonPin, INPUT);
//sets buttonPin as input
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (digitalRead(buttonPin) == HIGH) {
    digitalWrite(ledPin, HIGH);
  }
  else {
    digitalWrite(ledPin, LOW);
  }
}
```

Method 2: External Pull-Up Resistor

Instead of connecting the resistor to GND, we connect it to **5 V**.

When the button is not pressed, the resistor keeps the input **HIGH**.

Pressing the button connects the pin directly to **GND**, forcing it LOW.

Notice that the logic becomes **inverted**.

Wiring

- One side of the button → **GND**
- Other side → **Digital Pin**
- **10 kΩ resistor** between Digital Pin and **5 V**

Button	Pin State
Released	HIGH
Pressed	LOW

```
const int buttonPin = 2;
const int ledPin = 13;
```

```
void setup() {
  pinMode(buttonPin, INPUT);
//sets buttonPin as input
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (digitalRead(buttonPin) == LOW) {
    digitalWrite(ledPin, HIGH);
  }
  else {
    digitalWrite(ledPin, LOW);
  }
}
```

Method 3: Arduino's Internal Pull-Up Resistor

The ATmega328P microcontroller inside the Arduino Uno already contains an internal pull-up resistor (approximately **20–50 kΩ**, typically around **30 kΩ**). Instead of adding an external resistor, Arduino can enable this resistor in software.

This is done using

```
pinMode(buttonPin, INPUT_PULLUP);
```

The internal resistor connects the input pin to **5 V**, so the circuit behaves exactly like the external pull-up circuit.

> ✗ **COMMON MISTAKE**
>
> Writing `if (buttonState == HIGH)` with INPUT_PULLUP turns on the LED when the button is NOT pressed. Always remember: with INPUT_PULLUP, LOW = pressed.

Digital Input with digitalRead()

`digitalRead(pin)` returns either `HIGH` (1) or `LOW` (0) — an integer value. You almost always store the result in a variable before using it:

```
int buttonState = digitalRead(buttonPin);  // store the reading
if (buttonState == LOW) {                  // then use it
  // button is pressed (with INPUT_PULLUP)
}
```

Why store it? If you call `digitalRead()` twice in the same `if` statement, the button state might change between calls (button bouncing). Storing it once ensures you are testing the same reading throughout the condition.

3.5 The Serial Monitor

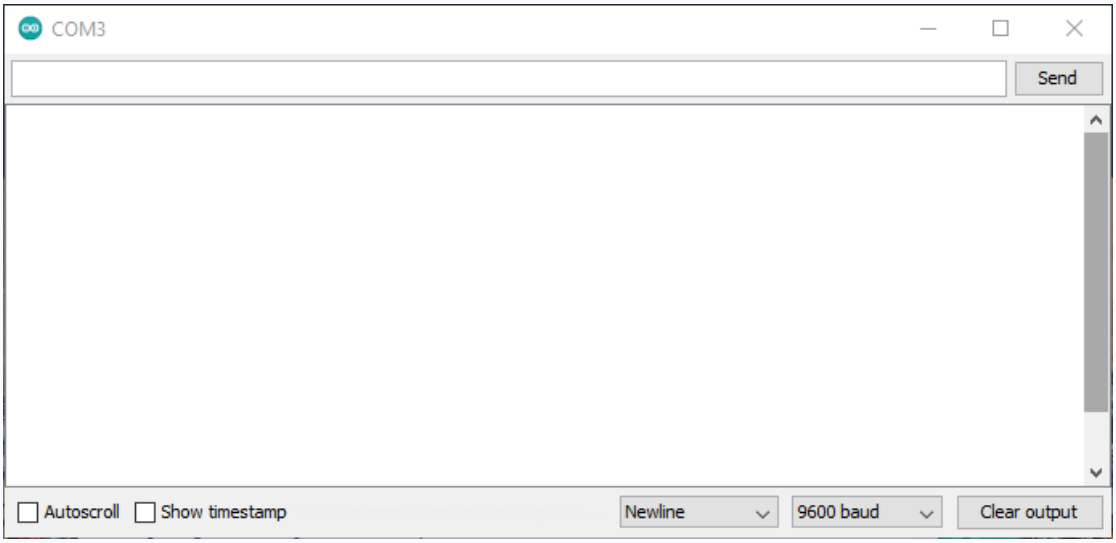
The Arduino has no built-in screen. The **Serial Monitor** is your window into the board's mind — a live text connection between the Arduino and your computer over the USB cable you are already using.

Engineers use the Serial Monitor for three purposes:

- **Debugging** — print variable values and status messages to find errors instantly, without any extra hardware.

- **Live sensor values** — watch readings update in real time as you interact with sensors (turning a knob, moving your hand near a sensor, pressing a button).
- **Understanding program flow** — print "Reached setup()" or "Button pressed!" to see exactly when each part of your code runs.

Figure 3.5: The Serial Monitor Interface



Setting It Up

6. In `setup()`, add: `Serial.begin(9600);`
7. Open the monitor: **Tools** → **Serial Monitor**.
8. Set the **baud rate dropdown** to **9600** — this must match the number in `Serial.begin()`. If they do not match, you see garbled symbols instead of text.

Baud rate is the communication speed in bits per second. 9600 is the workshop standard. Both the Arduino and the Serial Monitor must agree on this value, just like both ends of a phone call must use the same language.

Printing Commands

Serial.print() vs Serial.println()

```
Serial.print("Text");         // prints text, stays on the same line
Serial.println("Text");       // prints text, then moves to a new line
Serial.print(value);          // prints a number variable
Serial.println(value);        // prints a number, then new line

// Combining: print label and value on one line
Serial.print("Distance: ");   // label (no newline)
Serial.print(distance);       // value (no newline)
Serial.println(" cm");        // unit + newline
```

TIP

Serial.print() and **Serial.println()** differ by one line break. `print()` continues on the same line; `println()` (short for "print line") adds a newline character at the end, moving

subsequent output to the next line. Mixing them intentionally lets you put a label and a value on the same line: `Serial.print("Level: ");` then `Serial.println(sensorValue);`

Example — Watch a Button Press in Real Time (0.1 Basics → DigitalReadSerial)

Watching Button State in Serial Monitor

```
const int pushButton = 2;
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  // make the pushbutton's pin an input:
  pinMode(pushButton, INPUT);
}
// the loop routine runs over and over again forever:
void loop() {
  // read the input pin:
  int buttonState = digitalRead(pushButton);
  // print out the state of the button:
  Serial.println(buttonState);
  delay(1);        // delay in between reads for stability
}
```

Open the Serial Monitor and press the button. You will see the number change from 0 to 1 when pressed.

➤ BEYOND THE WORKSHOP

`delay()` is simple and works well for the early examples. But it has a serious limitation: **while the Arduino is inside a `delay()`, it cannot do anything else**. It cannot read a button press, cannot update an LED, cannot check a sensor. The program is completely paused.

`millis()` returns the number of milliseconds since the Arduino last powered on or reset. It is like reading a stopwatch that is always running.

The difference between `delay()`-based code and `millis()`-based code is the difference between blocking and non-blocking programming. Nearly all professional embedded code avoids `delay()` for exactly this reason. When you build a project that needs to do two things "at the same time" — blink an LED AND read a button, for example — `millis()` is the tool you reach for.

3.6 Conditional Logic

Real programs make decisions. The Arduino does this with the same logic you use every day:

if / else if / else Structure

```
if (condition) {
  // executes when condition is TRUE
} else if (otherCondition) {
  // executes if first was FALSE and this is TRUE
} else {
  // executes if nothing above was TRUE
}
```

You can chain as many else if clauses as you need.

The Button → LED Program (0.2 Digital → Button)

Button → LED

```
const int buttonPin = 2; // Pin connected to the push button
const int ledPin    = 13; // Pin connected to the built-in LED

void setup() {
  pinMode(buttonPin, INPUT);  // Set button as an input
  pinMode(ledPin, OUTPUT); // Set built-in LED as an output
}

void loop() {
  int buttonState = digitalRead(buttonPin);  // 1 = pressed, 0 = not pressed

  if (buttonState == HIGH) {       // HIGH means PRESSED
    digitalWrite(ledPin, HIGH);  // button down → LED on
  } else {
    digitalWrite(ledPin, LOW);   // button up → LED off
  }
}
```

The Critical = vs == Distinction

✗ COMMON MISTAKE

`if (buttonState = LOW)` — uses single =. This ASSIGNS the value LOW (which is 0, which is false) to buttonState and then tests the result. The LED will never turn on. This is a valid C statement that compiles without error but behaves unexpectedly.

`if (buttonState == LOW)` — uses double ==. This COMPARES buttonState to LOW. The condition is true when they are equal. Always use == for comparison inside if statements.

Memory aid: = is assignment (puts a value in); == is equality (asks "are these equal?"). A single = can never appear in a condition — if you find yourself writing one, it is almost always a bug.

Try the below given Reflex Timer Game as a fun extended activity:
https://projecthub.arduino.cc/rowan07/a-simple-reflex-game-dcd2c8

✓ KEY TAKEAWAYS

Every sketch has `setup()` (runs once) and `loop()` (runs forever). All variables that need to persist between loop() calls must be declared at the top of the sketch.

`digitalWrite()` controls outputs; `digitalRead()` reads inputs. Both use HIGH/LOW.

Always use `INPUT_PULLUP` for buttons — not `INPUT`. With INPUT_PULLUP, HIGH = not pressed, LOW = pressed.

The **Serial Monitor** reveals what a running program is doing — open it whenever you are debugging.

? QUICK REVIEW QUESTIONS

- Rewrite the Button → LED code using `INPUT_PULLUP`. What changes in the `if` condition, and why?
- Explain the difference between `=` and `==` in an `if` statement. Give an example of each and describe what each does.
- What does `digitalRead(2)` return? What type of value is it? What does each possible return value mean when INPUT_PULLUP is used?
- Why is the Serial Monitor useful even when no compile errors are shown?
- What does `Serial.begin(9600)` do, and where in the sketch must it be placed?
- What is the difference between `Serial.print("Value: ")` and `Serial.println(42)`? What does the Serial Monitor show if both are called one after the other?
- Explain why `delay()` prevents the Arduino from reading a button press while it is waiting. What is the solution?

➨ MINI ACTIVITIES

Floating Pin Demonstration: Wire a button to pin 2 using `INPUT` mode (NO pull-up). Open Serial Monitor. Wave your hand near the breadboard without touching it. Observe the readings. Then change to `INPUT_PULLUP` and observe the difference. Document what you see.

Toggle Challenge: Modify the button-LED program so each press TOGGLES the LED (press once = on, press again = off). Hint: use a `boolean` state variable. When the button is pressed (reads LOW), flip the state with `ledState = !ledState`, then use `digitalWrite(ledPin, ledState)`.

Three-LED Chaser: Wire three LEDs to pins 9, 10, 11 (each with a 220Ω resistor). Write a sketch using a `for` loop and an array to chase the light forward across the LEDs, then back.

Reaction Logger: Extend the reflex game to keep track of the best (lowest) reaction time across multiple rounds. Print the best time at the end of each round: "Best so far: X ms".

M4

Module 4

Analog Input, PWM, and the Serial Monitor

⏱ 4 Hours

Teach the Arduino to measure and produce in-between values. Sensors become meaningful when you can read a knob, sense light, and dim an LED.

4.1 The Digital World vs the Real World

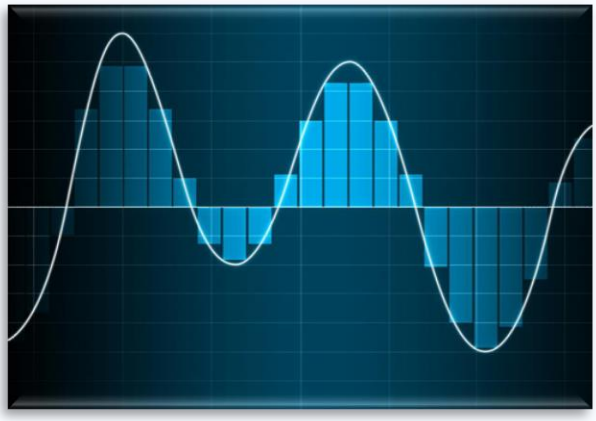
So far the Arduino has thought only in extremes: on or off, HIGH or LOW. The real world is rarely so binary — light fades gradually, a knob turns smoothly, sound rises and falls. This module teaches the Arduino to **measure** and **produce** in-between values.

A useful mental shortcut:

- **Digital ≈ integers.** Between 0 and 1 there are only two whole values: 0 and 1. A light switch: on or off.
- **Analog ≈ real numbers.** Between 0 and 1 there are infinitely many values: 0.5, 0.512, 0.51234… A dimmer switch: any brightness.

	Digital	Analog
States	Two only (HIGH / LOW)	Continuous (0V to 5V — infinite gradations)
Example	Light switch	Dimmer knob
Good for	Buttons, simple LEDs, on/off control	Brightness, knob position, sound, temperature
Arduino reading method	digitalRead() → 0 or 1	analogRead() → 0 to 1023

Figure 4.1: Analog vs Digital Signals



4.2 How the Arduino Reads Analog Inputs

The Arduino's microcontroller is a digital device — it only works with whole numbers. When an analog sensor produces a smoothly varying voltage, the Arduino must first convert that voltage into a number it can work with. This conversion is performed by the **ADC — Analog-to-Digital Converter**, hardware built directly into the ATmega328P chip.

On the Arduino Uno, the ADC converts any voltage between 0V and 5V into a whole number between **0 and 1023**.

Voltage at Pin	ADC Reading	Notes
0 V	0	Connected to GND
~1.25 V	~256	One quarter
~2.5 V (halfway)	~512	Dead centre
~3.75 V	~768	Three quarters
5 V	1023	Connected to 5V

You read an analog pin with:

```
analogRead(A0);   // returns a whole number from 0 to 1023
```

Only the six **analog pins A0–A5** support `analogRead()`. Digital pins cannot perform this conversion.
The ADC is 10-bit, meaning it produces 2^{10} = **1024 discrete steps,** which is why we obtain values from 0 to 1023.

Each step represents:
5V ÷ 1024 steps = **4.88 mV** (approximately 5 mV)

4.3 Component Spotlight: The Potentiometer

A **potentiometer** (pot) is an adjustable resistor — a knob you turn to produce a variable voltage. It is the simplest possible analog input. You have used one as a volume knob, a fan speed dial, or a brightness control.

Inside, a resistive strip runs from one end to the other, and a sliding contact (the **wiper**) moves along it as you turn the knob. The pot has three pins:

- One outer pin → **5V (VCC)**
- Other outer pin → **GND**
- Middle pin (wiper) → **analog input pin (A0)**

Turning the knob slides the wiper, changing the wiper voltage from 0V to 5V. The ADC reads this as 0 to 1023. The potentiometer is a self-contained voltage divider — the middle pin always sits between the voltages at the two outer pins, at a position determined by the knob angle.

Try It Yourself

Load **File → Examples → 01.Basics → AnalogReadSerial**, open the Serial Monitor at 9600 baud, and turn the knob slowly from one end to the other. Watch the numbers sweep from 0 to 1023 in real time.

Read Potentiometer to Serial Monitor

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int value = analogRead(A0); //analogRead(A0)reads analog values on pin A0
  Serial.println(value);
  delay(100);
}
```

4.4 Component Spotlight: The LDR

An **LDR (Light Dependent Resistor)**, also called a photoresistor, changes its resistance based on the amount of light it receives:

- **Bright light → low resistance** (few hundred ohms) → high voltage at junction → analogRead gives HIGH number
- **Darkness → high resistance** (tens of thousands of ohms) → low voltage at junction → analogRead gives LOW number

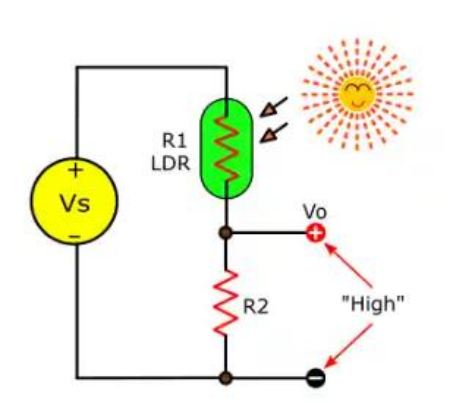
LDRs are the sensor technology behind automatic street lights, night lamps, camera auto-exposure systems, and smart blinds.

The Voltage Divider Circuit — Critical for the LDR

An analog pin measures **voltage**, not resistance. But the LDR changes its **resistance** with light. The question is: how do you convert a changing resistance into a changing voltage that the Arduino can read?

The answer is a voltage divider: two resistors in series between 5V and GND, with the junction between them connected to the analog pin. When one resistor changes (the LDR), the voltage at the junction changes.

Figure 4.4: LDR Voltage Divider Circuit



Why does this work? In a voltage divider, the voltage at the junction depends on the ratio of the two resistors:
V_out = V_supply × R2 / (R_LDR + R2)

When the LDR is in bright light: its resistance drops to a few hundred ohms. R2 (10kΩ) is now much larger than R_LDR. V_out is close to 5V → analogRead gives a high number.

When the LDR is in darkness: its resistance rises to tens of thousands of ohms. R_LDR is now much larger than R2. V_out is close to 0V → analogRead gives a low number.

This is why the LDR circuit needs a 10kΩ partner resistor — without it, the analog pin would only ever see 5V regardless of light level (the LDR alone, connected from 5V to A0, produces no voltage division). This is one of the most common beginner wiring mistakes for LDR circuits.

But the LDR Module we will be using will do it internally, and we only need to connect VCC and GND, and take input from A0 or D0 accordingly.

> ✗ **COMMON MISTAKE**
>
> Wiring the LDR with only two connections — one leg to 5V and one leg to A0, with no second resistor — gives a voltage divider with an infinite lower resistor. The formula gives V_out = 5V at all times. The Arduino reads ~1023 in all lighting conditions and the system does not respond to light changes.
>
> Fix: always add a 10kΩ resistor from the A0 junction to GND. The LDR goes from 5V to the junction. The 10kΩ goes from the junction to GND. A0 reads the junction voltage.

Using the Serial Monitor with Analog Sensors

The Serial Monitor is your most valuable debugging tool when working with analogue sensors. Seeing live numbers from analogRead() tells you immediately whether the sensor is wired correctly, what value it reads at each physical position, and what threshold values to use in your if/else conditions. Always calibrate with the Serial Monitor before writing output logic.

- **Step 1:** Add Serial.begin(9600) to setup(). Open Serial Monitor (Ctrl+Shift+M). Set baud rate to 9600.
- **Step 2:** In loop(): int sensorValue = analogRead(A0); Serial.println(sensorValue); delay(100);
- **Step 3:** Move the potentiometer (or cover the LDR). Record the min and max values you observe. Use these as your map() input range.
- **Step 4:** Choose a threshold for your if/else condition based on observed values. Replace the hardcoded 512 with your calibrated threshold.

4.5 PWM: Faking Analog Output

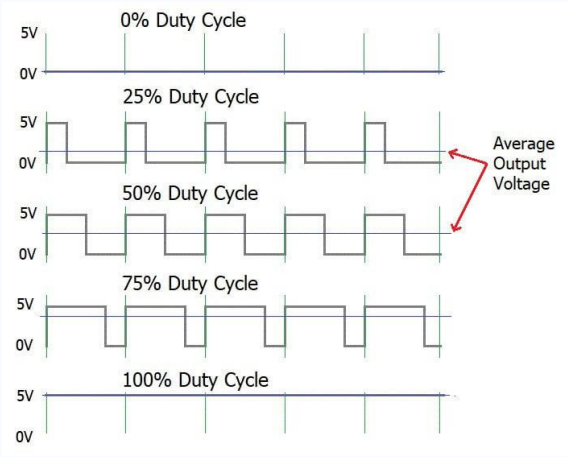
We can read analog values from sensors. But can the Arduino produce analog output? A digital pin can only output fully ON (5V) or fully OFF (0V) — there is no true 2.5V setting available on a digital output.

The clever solution is **PWM — Pulse Width Modulation**.

The Idea

PWM creates the illusion of an analog output by switching a pin ON and OFF very rapidly — hundreds of times per second. By varying how much of each cycle is spent ON (called the **duty cycle**), you control the average power delivered to the load.

Figure 4.5: PWM Duty Cycle



- **Wide pulse** (long ON time, high duty cycle) → high average power → LED appears bright.
- **Narrow pulse** (short ON time, low duty cycle) → low average power → LED appears dim.

The switching happens at approximately 490 Hz to 980 Hz on the Uno — much too fast for your eye to detect the flicker. Your brain integrates the rapid ON/OFF into a perceived steady brightness at the duty cycle's average level, exactly as a cinema's 24 frames per second blend into perceived motion.

Using analogWrite()

analogWrite() Examples

```
analogWrite(pin, value);  // value: 0 (fully off) to 255 (fully on)

// Examples:
analogWrite(9, 0);    // LED off
analogWrite(9, 64);   // LED dim (~25% brightness)
analogWrite(9, 128);  // LED at half brightness
analogWrite(9, 255);  // LED at full brightness
```

> ⚠ **IMPORTANT**
>
> PWM only works on the six pins marked with **~** on the Uno: pins **3, 5, 6, 9, 10, 11**. Calling `analogWrite()` on any other digital pin produces no dimming — the pin either stays HIGH or behaves unpredictably. Always connect your dimming LED or motor to a ~ pin.

Why only six pins? The ATmega328P has three built-in hardware **timers**, each driving two PWM channels. Six channels total → six PWM-capable pins. The other digital pins have no timer hardware, so they cannot generate PWM signals independently.

PWM beyond LEDs: The same technique that fades your LED controls fan speeds, motor speeds, and dimmable lights everywhere in real electronics. A motor connected to a PWM pin runs faster at high duty cycles and slower at low ones. The technique is identical.

Try it youself:

analogWrite() to control LED Brightness

```
int ledPin = 9; // Must be a PWM pin (look for the ~ symbol on your board)

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // 0 = OFF (Fully dark)
  analogWrite(ledPin, 0);
  Serial.println("0");
  delay(2000);

  // 64 = DIM (Faint glow)
  analogWrite(ledPin, 64);
  Serial.println("64");
  delay(2000);

  // 128 = HALF (Medium brightness)
  analogWrite(ledPin, 128);
  Serial.println("128");
  delay(2000);

  // 192 = BRIGHT (Strong glow)
  analogWrite(ledPin, 192);
  Serial.println("192");
  delay(2000);

  // 255 = MAX (Full brightness)
  analogWrite(ledPin, 255);
  Serial.println("255");
  delay(2000);
}
```

4.6 Serial Monitor for Sending Data

The **Serial Monitor** is not only used to display data from the Arduino, but also to **send data from the computer to the Arduino**. This allows you to control your program without changing the code every time. You can type numbers, letters, or words into the Serial Monitor, and the Arduino reads them through the USB connection.

For receiving data on Arduino, serial communication must first be initialized in the `setup()` function using `Serial.begin(9600);`

Before reading data, the Arduino should check whether any characters are available.

```
if (Serial.available() > 0) {
    // Read incoming data
}
```

The most commonly used functions are:

Function	Description
Serial.read()	Reads a single character from the Serial Monitor.
Serial.parseInt()	Reads an integer entered by the user.
Serial.readString()	Reads an entire string of text until timeout.

Example: Controlling LED Brightness from the Serial Monitor

In this example, the user enters a value between **0 and 255** in the Serial Monitor. The Arduino reads the value and uses **PWM** to adjust the brightness of an LED.

Serial Monitor-Controlled LED Brightness

```
// 1. Tell the Arduino which pin connects to the LED
const int ledPin = 3;

void setup() {
  // 2. Open the communication channel to the computer
  Serial.begin(9600);

  // 3. Configure the LED pin to send out power
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // 4. Check if the user has typed anything into the computer
  if (Serial.available() > 0) {

    // 5. Read the full number entered by the user
    int brightness = Serial.parseInt();
    // If brightness is over 255, force it to stay at 255
      brightness = constrain(brightness, 0, 255);

    // 6. Send that exact number to change the LED brightness
    analogWrite(ledPin, brightness);

    // 7. Print feedback so the user knows the Arduino received it
    Serial.print("Setting brightness to: ");
    Serial.println(brightness);
  }
}
```

constrain(value, min, max)` clamps a value so it never falls outside a specified range:

constrain() Function

```
brightness = constrain(brightness, 0, 255);  // prevent analogWrite()
receiving out-of-range value
```

4.7 Potentiometer-Controlled Brightness

This project ties analog input to PWM output: turn the knob, and the LED brightness follows in real time.

Potentiometer-Controlled LED Brightness

```
const int potPin = A0;
const int ledPin = 9;    // MUST be a PWM (~) pin
```

```
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
// 1. Read the potentiometer value (Range is 0 to 1023)
  int potValue  = analogRead(potPin);

  // 2. Translate (map) that value to a brightness level
  // We map the pot's input (0-1023) into the LED's PWM output (0-255)
  int brightness = map(potValue, 0, 1023, 0, 255);

  // Note: Because 1024 divided by 4 is exactly 256, you could also
  // just do this: brightness = potValue / 4;

  brightness = constrain(brightness, 0, 255);   // clamp (safety)
  // 3. Command the LED to light up at that specific brightness
  analogWrite(ledPin, brightness);

  // 4. Print the values to the Serial Monitor
  Serial.print("Pot: ");
  Serial.print(potValue);
  Serial.print("  →  Brightness: ");
  Serial.println(brightness);

  // 5. Wait a tiny bit (50 milliseconds) before reading the knob again
  delay(50);
}
```

The central new function here is **`map()`**:
`map(value, fromLow, fromHigh, toLow, toHigh)` — rescales a number from one range to another. The potentiometer produces values from 0 to 1023, but `analogWrite()` needs values from 0 to 255. `map()` translates between them proportionally, similar to how you convert marks to out of 100 while calculating percentage. This is one of the most useful utility functions in Arduino programming.

The complete data flow in this project:

Potentiometer → analogRead() → raw value (0–1023) → map() → brightness value (0–255) → analogWrite() → LED brightness

> **TIP**
>
> Use `constrain()` after `map()` whenever sensor noise might push the mapped value outside the acceptable range. `analogWrite()` only accepts values from 0 to 255 — passing it a value outside this range causes undefined behaviour.

4.8 Troubleshooting Analog Circuits

Symptom	Cause	Fix
LDR reads same value in all lighting	Missing 10kΩ voltage divider resistor — only LDR connected	Add 10kΩ resistor from A0 to GND. LDR goes from 5V to A0.
Readings jump ±20 even with no movement	Normal ADC noise on analog input	Average multiple readings: sum 5 readings, divide by 5. Or use constrain()
Stuck at 0 or 1023	Sensor wired to wrong pin, or missing connection	Verify A0–A5; check both sensor connections with a multimeter if available
LED brightness does not change	LED on a non-PWM pin	Move LED to a ~ pin (3, 5, 6, 9, 10, 11)
Garbled Serial text	Baud rate mismatch	Match Serial.begin() value to the dropdown in Serial Monitor (9600)
map() gives wrong result	Integer truncation — map() rounds down	Use constrain() after map(). For accurate results with small ranges, scale up first

✓ KEY TAKEAWAYS

The real world is **analog** (continuous); the Arduino is **digital**. The ADC bridges this gap by converting 0–5V into a number 0–1023.

The ADC is **10-bit** → 1024 steps → approximately 4.88mV per step.

An **LDR** needs a **10kΩ voltage divider resistor** to function — without it the analog pin reads a constant high value.

PWM fakes analog output by switching at high speed. `analogWrite(pin, 0–255)` works only on ~ pins (3, 5, 6, 9, 10, 11).

`map()` rescales values between two ranges. Use `constrain()` after map() to prevent out-of-range values.

A **potentiometer** is a self-contained voltage divider — no extra resistor needed.

? QUICK REVIEW QUESTIONS

- Why does the Arduino need an ADC to read a light sensor? What would happen if the Arduino tried to read resistance directly?
- What range of numbers does analogRead() return? Which pins on the Uno support it?
- Explain in one sentence how PWM produces a "half-brightness" LED.
- What does `map(sensor, 0, 1023, 0, 180)` do? Give a real example of where you would use this.
- An LDR has resistance 500Ω in bright light and 50kΩ in darkness. It is wired in series with a 10kΩ resistor from 5V to GND. Calculate the voltage at the junction (A0 pin) in each condition.
- Which Uno pins support `analogWrite()`? How are they marked on the board? Why can't other pins do this?
- The Serial Monitor shows the same value regardless of how bright or dark you make the LDR environment. What is the most likely cause?
- What does `constrain(value, 0, 255)` do and when should you use it with `map()`?

➨ MINI ACTIVITIES

Knob Dimmer: Build the potentiometer-controlled brightness circuit. Find the knob position that produces exactly half brightness (analogWrite value = 128). Verify by reading the Serial Monitor.

LDR Calibration: Record analogRead() values for 5 different lighting conditions (see calibration table). Choose a threshold that reliably separates "dark" from "light" for your environment. Upload the night light code using your chosen threshold.

Mood Buzzer: Map the LDR reading to a buzzer frequency using tone(). Cover and uncover the LDR. Use: `tone(8, map(analogRead(A0), 0, 1023, 200, 2000))`. Describe what you hear.

M5

Module 5

Servo Motors and Motion Control

⏱ 3 Hours

Give your project the power to move. Learn how servo motors work, use the Servo library, and create sensor-driven motion.

5.1 Why Movement Matters

Until now your projects have sensed and signalled. Now they will move. Movement is what turns electronics into robotics — machines that do not just react, but physically act on the world.

A complete automated system needs both halves of the loop working together:

- **Sensors detect** — they read light, distance, touch, temperature, and convert these into data.
- **Motors act** — they receive commands and produce physical motion: turning, lifting, opening.

Together, sensors gather information and motors execute action, creating systems that respond to the world automatically. A gate that opens for an approaching car, a lid that lifts when a hand approaches, an arm that tracks the sun — all these need motion under precise control.

5.2 Component Spotlight: The Servo Motor

A **servo motor** is a motor you can command to a precise angle — and it holds that position indefinitely. You tell it exactly where to go (say, 90 degrees), and it moves there and stays.

	DC Motor	Servo Motor	Stepper Motor
Motion type	Spins continuously	Precise angle (0°–180°)	Precise steps (full rotation)
Position control	None	Built-in (feedback loop)	Counted steps
Speed control	Yes (PWM)	Limited (write speed)	Yes (step rate)
Typical use	Wheels, fans, drills	Arms, gates, robotic joints	3D printers, CNC machines
Driver complexity	H-bridge required	Direct from Arduino	Dedicated driver IC

A Servo Motor has three pins – Red for VCC, Brown for GND, and Orange for SIGNAL.

Internally, a servo is not just a motor. It is a motor + gear train + position sensor (potentiometer) + control circuit, all packaged together. The control circuit continuously compares the commanded angle to the actual angle (measured by the internal potentiometer) and drives the motor to eliminate the difference. This is called **closed-loop control** — the motor checks its own position and corrects errors automatically.

The **SG90 micro servo** in your kit is the world's most common starter servo: lightweight, forgiving, and well-documented.

A servo's position is measured in degrees, from 0° to 180° on the SG90:

How a Servo Actually Works: The PWM Signal

The `Servo.write(angle)` function hides the underlying mechanism, but understanding it helps you debug servo problems.

A servo responds to the **pulse width** of a control signal sent 50 times per second (50 Hz). The width of each pulse determines the servo's commanded angle:

The `Servo.h` library calculates the correct pulse width for any angle from 0 to 180 and generates that signal automatically. This is the power of libraries: complex timing code, written once by experts, reused by everyone.

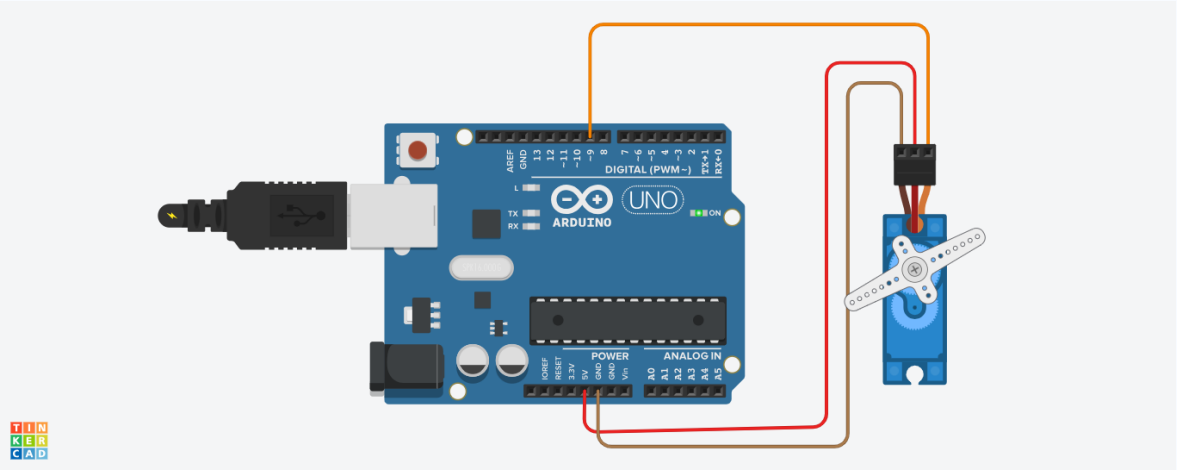
> ⚠ **IMPORTANT**
>
> **Power supply:** The SG90 draws approximately 150mA running and up to 700mA when stalled. A single servo is fine powered from the Arduino's 5V pin (which can supply ~800mA from USB). However, two or more servos, or any servo under load, may draw more than the Arduino can supply, causing the board to reset. For multiple servos: use an external 5V power supply connected directly to the servo power pins, with GND shared with the Arduino.

5.3 The Servo Library

Generating precise 50Hz pulses by hand would require complex timer code. The **Servo library** handles all of this automatically — it is pre-written, tested code that you unlock with one line.

What is a library? A library is a collection of functions written and tested by someone else that you borrow by including it in your sketch. The Servo library, the `#include <Servo.h>` line, and the `Servo myServo;` object are all part of this borrowing process. When you call `myServo.write(90)`, you are using code that another programmer wrote to generate the correct 1.5ms pulses.

Figure 5.1: Servo connections



Basic Servo Control — Three Positions

```
#include <Servo.h>      // load the Servo library
```

```
Servo myServo;          // create a Servo object named myServo
void setup() {
  myServo.attach(9);    // Attach servo to digital pin 9
}
void loop() {
  myServo.write(0);     // move to 0 degrees (full left)
  delay(2000);
  myServo.write(90);    // move to 90 degrees (centre)
  delay(2000);
  myServo.write(180);   // move to 180 degrees (full right)
  delay(2000);
}
```

Line-by-Line Explanation

- `#include <Servo.h>` — a **compiler directive** (starts with #) that tells the compiler to include the Servo library's code in your sketch.
- `Servo myServo;` — creates a **Servo object**. Think of it as a variable that represents your physical servo, with built-in functions (`attach()`, `write()`, `detach()`).
- `myServo.attach(9)` — links the servo object to the signal pin. The library starts generating 50Hz pulses on pin 9.
- `myServo.write(angle)` — commands the servo to move to the specified angle (0–180). The library calculates the correct pulse width automatically.
- `myServo.detach()` — stops generating pulses. Use this when you want the servo to relax rather than hold its position (reduces battery drain and heat in the servo).

5.4 The for Loop Applied to Servo Sweep

Sweeping a servo from 0 to 180 one degree at a time would require 180 separate `write()` calls if done manually. A `for` loop does it in three lines:

Servo Sweep using for Loop

```
// SERVO SWEEP — smooth motion from 0° to 180° and back
#include <Servo.h>

Servo myServo;

void setup() {
  myServo.attach(9);
}

void loop() {
  // Sweep from 0° to 180° (increasing)
  for (int angle = 0; angle <= 180; angle++) {
    myServo.write(angle);  // angle variable directly used as position
    delay(10);             // pause 10ms between each degree — smooth motion
  }

  // Sweep from 180° back to 0° (decreasing)
  for (int angle = 180; angle >= 0; angle--) {
    myServo.write(angle);
```

```
    delay(10);
  }
}
```

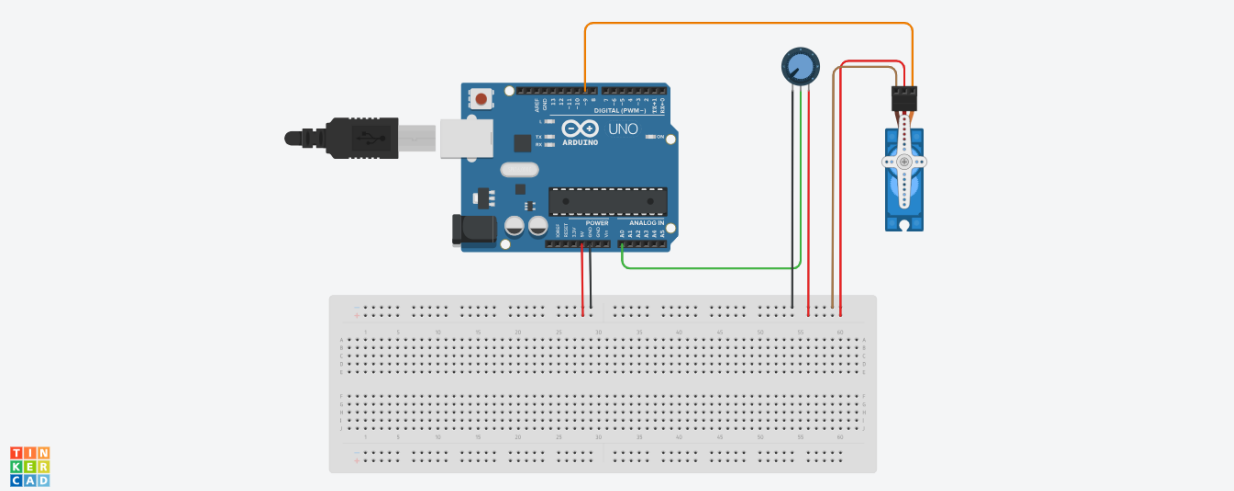
Notice how the loop variable `angle` is used directly as the servo position. This is a very common and elegant pattern: the variable tracking loop progress (0 to 180) is simultaneously the value being sent to the servo. Reducing `delay(10)` makes the sweep faster; increasing it makes it slower.

5.5 Sensor-Controlled Motion

The real power of a servo comes when a sensor — not a fixed program — drives it. This is **environment-driven automation**: the machine reacts to the world, not just to a script.

Potentiometer-Controlled Servo

Figure 5.5: Potentiometer-Controlled Servo Circuit



Potentiometer → Servo (Proportional Control)

```
#include <Servo.h>

Servo myServo;
const int potPin = A0;

void setup() {
  myServo.attach(9);
}

void loop() {
  int potValue = analogRead(potPin);            // read knob: 0-1023
  int angle    = map(potValue, 0, 1023, 0, 180); // scale to: 0-180
  angle = constrain(angle, 0, 180);             // clamp to safe range

  myServo.write(angle);
```

```
  delay(15);
}
```

Notice `map()` returning here, now converting the potentiometer's 0–1023 range into the servo's 0–180 range. The knob becomes a direct, intuitive human-machine interface — a joystick for the servo.

The same idea can be used with any Analog input.

⚙ PRACTICAL CONNECTION

Sensor-controlled motion is the foundation of robotics and automation. Sensors provide awareness; the Arduino provides decision-making; servos provide action. The four-step model — sense → read → map → act — appears in everything from servo arms to self-driving vehicles.

Symptom	Cause	Fix
Servo jitters continuously even at rest	Power supply insufficient or noisy	Try powering servo from external 5V supply; add 100µF capacitor across servo power pins
Servo does not reach 0° or 180°	Mechanical end-stop; physical range narrower than 0–180	Use constrain() to limit range; find physical limits with calibration activity
Servo makes buzzing sound when holding position	Load too heavy for SG90 (stall condition)	Reduce the load or use a higher-torque servo
Servo twitches when loop() runs fast	Sending write() commands too rapidly	Add a short delay (10–15ms) between write() calls
#include <Servo.h> causes error	Library not installed (unusual — usually pre-installed)	Tools → Manage Libraries → search "Servo" → Install

✓ KEY TAKEAWAYS

A servo motor moves to a precise, held angle (0°–180°). Internally it is a motor + gears + position sensor + controller.

Servos respond to PWM pulse width: 1ms = 0°, 1.5ms = 90°, 2ms = 180°, at 50Hz.

The **Servo library** (`#include <Servo.h>`) handles pulse generation. Call `attach()` then `write(angle)`.

`map()` converts sensor readings (0–1023) into servo angles (0–180). Always follow with `constrain(angle, 0, 180)`.

A `for` loop makes servo sweeping elegant: the loop variable IS the angle.

Sensor-controlled motion — where the sensor drives the servo — is the core of robotics and automation.

? QUICK REVIEW QUESTIONS

- What is the key difference between a servo motor and a DC motor?
- What does `myServo.attach(9)` do? What would happen if you called `myServo.write(90)` before `attach()`?
- Through what range of angles can the SG90 servo move? What happens if you write(200) to it?
- Why do we use `map()` when driving a servo from a potentiometer? What would the servo do without it?
- Identify the three servo wires and where each connects on the Arduino.
- Explain the servo PWM signal: what happens at a 1ms pulse vs a 2ms pulse?
- Write a for loop that sweeps a servo from 45° to 135° and back, with 15ms between each degree.
- Why might multiple servos need a separate power supply rather than the Arduino's 5V pin?

➨ MINI ACTIVITIES

Servo Calibration: Call myServo.write(0) and mark the physical position of the horn. Then write(180). Measure the actual angle with a protractor. Does the servo rotate exactly 180°? Document the actual range.

Smooth Sweep: Build the servo sweep using a for loop. Vary the delay between 5ms, 15ms, and 50ms. Describe the difference in motion quality.

Knob Arm: Build the potentiometer-controlled servo. Open Serial Monitor. Verify that at one end of the knob the angle reads ~0, at the other end ~180.

Light Follower: Wire an LDR + 10kΩ divider on A0 and the LDR-controlled servo code. Move a desk lamp to the left and right of the sensor. Describe how the servo responds.

Module 6

M6 Ultrasonic Sensors and Automation

⏱ 3 Hours

Give your machine the ability to "see" obstacles using sound. Master the HC-SR04, measure distance, and build automation systems that respond to proximity.

6.1 How Machines See with Sound

How does a machine detect an obstacle without a camera or eyes? With sound. The same principle nature invented millions of years ago for bats and dolphins.

Sound is a vibration that travels through air as a wave. In air at room temperature (~20°C), sound travels at approximately **343 metres per second** (about 0.034 centimetres per microsecond).

Echolocation: A bat emits a burst of ultrasonic sound and listens for the echo that bounces back from nearby objects. The time between emission and echo tells the bat how far away the object is. The HC-SR04 uses exactly this principle.

Ultrasonic means "beyond human hearing." Humans hear frequencies from about 20 Hz to 20,000 Hz. The HC-SR04 operates at **40,000 Hz (40 kHz)** — high enough to avoid confusion with environmental sounds, but ideal for clean, precise distance measurement.

6.2 Component Spotlight: The HC-SR04

Figure 6.1: HC-SR04 Ultrasonic Sensor

Pin	Direction	Role	Connect To
VCC	Power input	Provides 5V operating power	5V pin on Arduino
TRIG	Input (from Arduino)	Arduino sends a 10µs HIGH pulse here to start a measurement	Any digital output pin (e.g. pin 9)
ECHO	Output (to Arduino)	Goes HIGH for the duration of the round-trip echo time	Any digital input pin (e.g. pin 10)

GND	Power reference	Ground connection	GND pin on Arduino

Range: approximately 2 cm to 400 cm, with accuracy near ±3 mm under ideal conditions.

Understanding delayMicroseconds()

The trigger pulse sent to the HC-SR04 must be exactly 10 microseconds long. A microsecond is one millionth of a second (1µs = 0.000001 s). The standard `delay()` function works in milliseconds — its minimum is `delay(1)` which is 1000 microseconds. For the HC-SR04 trigger, you need much shorter pauses.

`delayMicroseconds(us)` works exactly like `delay()` but accepts a value in microseconds:

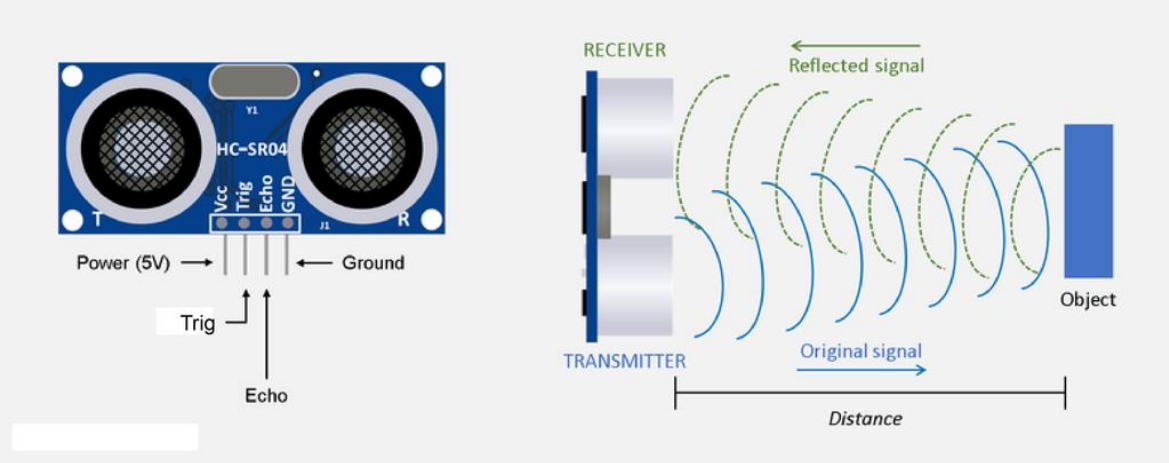
delay() vs delayMicroseconds()

```
delay(1000);                // pause for 1000 ms = 1 second
delay(1);                   // pause for 1 ms = 1000 microseconds
delayMicroseconds(10);      // pause for 10 microseconds = 0.01 ms
delayMicroseconds(2);       // pause for 2 microseconds = 0.002 ms
```

For the HC-SR04: sending a 10µs HIGH pulse and a brief 2µs LOW pulse before it creates the clean trigger signal the sensor requires.

6.3 How the HC-SR04 Measures Distance

Figure 6.2: HC-SR04 Echo Timing Sequence



The measurement process runs in three automatic steps:

1. **Trigger:** The Arduino sends a 10µs HIGH pulse to the TRIG pin. This commands the sensor to start a measurement.
2. **Emit:** The sensor emits eight ultrasonic pulses at 40kHz. These travel outward from the transmitter.
3. **Measure:** The pulses hit an object and reflect back. The ECHO pin goes HIGH immediately after the burst is sent, and goes LOW when the echo returns. The duration of the ECHO HIGH is equal to the round-trip travel time.

The distance formula:

Distance = (Echo time × speed of sound) ÷ 2

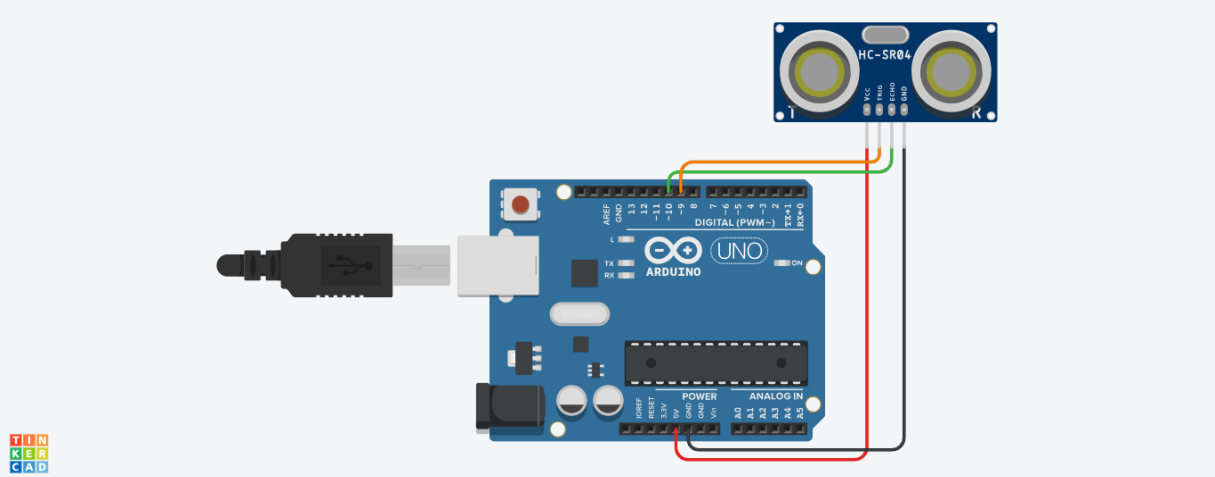
In Arduino-friendly units (where sound travels 0.034 cm per microsecond):

Distance (cm) = (Echo time in µs × 0.034) ÷ 2

We divide by 2 because the sound travels to the object and back — we only want the one-way distance.

Reading Distance in Code

Figure 6.4: HC-SR04 Wiring to Arduino



HC-SR04 Distance Reader — Complete Code

```
const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);  // TRIG sends signal — it is an output
  pinMode(echoPin, INPUT);   // ECHO receives — it is an input
}

void loop() {
  // --- STEP 1: Send a clean 10µs trigger pulse ---
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);          // Clear the trigger pin
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);         // HIGH for exactly 10 microseconds
  digitalWrite(trigPin, LOW);    // end the trigger pulse

  // --- STEP 2: Measure how long ECHO stays HIGH ---
  long duration = pulseIn(echoPin, HIGH);  // returns µs as long integer
```

```
  // --- STEP 3: Convert time to distance ---
  float distance = duration * 0.0343 / 2;   // cm

  // --- STEP 4: Handle out-of-range readings ---
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  }

  delay(200);  // wait between measurements
}
```

Explaining pulseIn()

`pulseIn(pin, HIGH)` is a built-in Arduino function that measures the duration of a pulse on a digital pin. It waits for the specified pin to go HIGH, starts a timer, waits for it to go LOW again, stops the timer, and returns the elapsed time in **microseconds** as a `long` integer.

Important behaviours of `pulseIn()`:

- Returns 0 if no pulse arrives within the timeout period (~1 second by default). Always check for 0 before using the result.
- Returns a `long` value — always use `long` (not `int`) for duration, and `float` for distance variables when working with the HC-SR04. An `int` on the Arduino Uno can only hold values up to 32,767; echo times can exceed this for distant objects, causing integer overflow (the computed distance becomes a huge wrong number).

6.4 HC-SR04 Limitations and Edge Cases

The HC-SR04 is reliable within its specifications, but beginners frequently encounter puzzling readings caused by these known limitations:

Limitation	Effect	How to Handle
Minimum range ~2cm	Objects closer than 2cm return 0 or wildly wrong values	Check for duration == 0 or distance < 2; discard these readings
Maximum reliable range ~200cm	Accuracy degrades significantly beyond 2m (sensor spec is 4m)	For reliable measurements, stay within 200cm
Detection angle ~15° cone	Objects at sharp angles may not reflect the echo back to the receiver	Point sensor directly at the target; ensure the target is larger than the cone at that distance
Temperature effect on sound speed	0.034 cm/µs assumes ~20°C; errors grow at extreme temperatures	For precision: use 0.0331 + 0.0006×(temp_celsius) cm/µs
Cross-talk between two sensors	If two HC-SR04 sensors face each other, they receive each other's pulses	Angle sensors away from each other; trigger them at different times

Hard smooth surfaces may miss	Smooth or angled hard surfaces deflect the echo sideways	Use textured or flat-perpendicular targets

6.5 Obstacle Detection and Automation

Once you can measure distance, automation is just a matter of comparing the reading against a **threshold** — exactly the same IF-THEN logic you used earlier.

Basic Threshold Detection

```
if (distance < 15) {
  // Object within 15 cm — ALERT
  digitalWrite(ledPin, HIGH);
  digitalWrite(buzzerPin, HIGH);
} else {
  // Safe distance — clear
  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);
}
```

Module 6 Troubleshooting

Symptom	Cause	Fix
Distance always 0	No echo returned; ECHO pin not reading	Check TRIG is OUTPUT, ECHO is INPUT; ensure object is within range and facing sensor
Distance reads huge number (millions)	Integer overflow: using int instead of long	Change int duration to long duration throughout the sketch
Readings jump ±20cm randomly	Nearby hard wall reflecting signal; electrical noise	Move sensor away from walls; ensure stable 5V supply
Sensor does not trigger	Trigger pulse too short or missing	Verify delayMicroseconds(10) on the HIGH pulse, not delay(10)
Object at close range reads 0	Object closer than 2cm minimum	Move object further away; check for 0 and treat as "too close"
Two sensors interfering	Cross-reception of each other's pulses	Angle sensors apart; trigger them alternately with separate timing

✓ KEY TAKEAWAYS

The HC-SR04 measures distance using ultrasonic echolocation: emit → travel → reflect → measure echo time.

TRIG receives a 10µs trigger pulse from the Arduino. **ECHO** reports the round-trip travel time.

`delayMicroseconds(10)` creates a 10µs pulse. `pulseIn(echoPin, HIGH)` measures the echo in microseconds.

Distance (cm) = (echo time in µs × 0.034) ÷ 2. Always use `long` — not `int` — for these values.

Always check for `duration == 0` (no echo returned) before calculating distance.

Comparing distance to thresholds with `if/else if/else` creates automation systems: parking alerts, intrusion detectors, automatic lids.

? QUICK REVIEW QUESTIONS

- Why is the HC-SR04 called an "ultrasonic" sensor? What frequency does it use, and why is this frequency chosen?
- What is the role of the TRIG pin versus the ECHO pin?
- Why do we divide the calculated distance by 2?
- What does `pulseIn(echoPin, HIGH)` return? What type should the receiving variable be?
- Write a complete trigger-and-measure sequence (5 lines of code) for the HC-SR04.
- An object is 30cm away. Calculate the expected echo time in microseconds. Show your working.
- Describe two limitations of the HC-SR04 that can cause unexpected readings and how to handle each.
- In the parking alert code, how does the beep interval change as the object approaches? What millis() pattern achieves non-blocking beeping?
- Describe the sense-decide-act loop of the smart dustbin system.

➨ MINI ACTIVITIES

Distance Display: Build the basic HC-SR04 distance reader. Move your hand at 10cm, 20cm, 50cm, and 100cm. Record the displayed values and calculate the percentage error at each distance.

Proximity Alarm: Add a buzzer on pin 8. Sound it when distance < 20cm. Verify the alarm triggers and stops as you move your hand.

Variable Beep Rate: Implement the parking alert code with three zones and a non-blocking beep rate that increases as distance decreases. Test with objects at various distances.

Precision Test: Measure a known distance of exactly 20.0cm with a ruler, then record 10 consecutive readings from the sensor. Calculate the mean and the range (max−min). Discuss the sensor's accuracy and repeatability.

M7

Module 7

Mini Project and Showcase

⏱ 5 Hours

Design, build, and demonstrate a smart system of your own. Everything you have learned — sensors, actuators, programming, the sense-decide-act loop — comes together here.

7.0 Team Formation and Roles

You will work in teams of 2–3 students. Suggested roles for team organisation:

Role	Primary Responsibilities	Must Also Know
Hardware Lead	Breadboard wiring, circuit assembly, component verification	The complete circuit diagram; how to read troubleshooting tables
Software Lead	Writing, uploading, and debugging the sketch	The pin assignments in the Hardware Lead's wiring
Documentation Lead	Project documentation template, showcase demo script, monitoring Serial output during testing	Enough code to read and explain it during the demo

> ⚠ **IMPORTANT**
>
> Roles guide collaboration, not division. Every team member must understand every part of the project. If only the Software Lead can explain the code during the showcase, the team is not ready. Every member should be able to answer any question about any part of the project.

7.1 Project Workflow — Build Incrementally

Follow these steps in order:

- **Choose and define.** Fill in all three fields: Input (sensor and pin), Processing (the decision rule in plain English), Output (actuator and pin). If you cannot complete all three before touching any hardware, discuss until you can.
- **Plan the wiring.** Sketch which component goes to which pin before touching the breadboard. Use the Pin Allocation table below. Get it checked by your instructor or partner.
- **Build incrementally.** Connect and test the sensor first — get a reading in the Serial Monitor. Then connect the actuator separately and test it with a fixed `write()` or `digitalWrite()`. Only then integrate them with the decision logic.

- **Code in stages.** Sensor-reading code first, verify it prints sensible values. Then add the decision logic. Then add actuator control. Never write the full program at once and hope it works.
- **Test with real data.** Use Serial Monitor readings to find the correct threshold value. Never guess — always measure.
- **Showcase.** Demonstrate your working system and explain its sense-decide-act loop to at least one other team.

7.2 Your AI Coding Assistant

As you design and build your project, you have a powerful extra tool available: AI coding assistants. These tools can help you write Arduino code, explain how a function works, find bugs, and suggest improvements — available instantly on your phone or laptop.

What AI Assistants Can Help With

- **describe what you want (e.g., "blink an LED when the distance is less than 20cm") and the AI can generate a starting sketch.**
- **ask "what does pulseIn() do?" and receive a clear, plain-language explanation with an example.**
- **paste your code and error message and ask the AI to explain what is wrong and why.**
- **ask "how can I make this code more efficient?" and receive concrete suggestions.**
- **faster than searching the Arduino documentation.**

Good Prompts vs Bad Prompts

The quality of the AI's response depends on how specifically you ask. Vague prompts give vague answers. Specific prompts that include your code, your pin numbers, and your exact goal get useful responses.

GOOD Prompts — Be Specific

```
GOOD: "Write modular Arduino code to control a PWM LED (pin 5)
      using an LDR (pin A0). Map the LDR range 1023-0 to PWM
      range 0-255, so the LED brightens as the LDR reading drops."

GOOD: "Explain and Fix this error in my loop() function: [paste code here]"

GOOD: "Explain what millis() does and give a simple example."

GOOD: "How do I read an ultrasonic sensor and print distance in cm?"

BAD:  "Write me a code to vary brightness."
BAD:  "Fix my code."
BAD:  "Make it work better."
```

AI-Assisted Debugging

- Paste your error message directly into the AI prompt — do not paraphrase it.

- Ask the AI to explain the bug, not just fix it. Understanding the cause prevents the same mistake next time.
- Always review and understand the AI's suggested changes before accepting them.
- Test the fix on your actual hardware. AI can be wrong about hardware-specific behaviour.
- Verify AI output — AI code can be syntactically correct but logically wrong. Run it and check the result.

What AI Cannot Replace

AI is a powerful tool, but you are the engineer. There are things AI simply cannot do for you:

- Understand your physical wiring — the AI cannot see your breadboard.
- Debug loose connections — only your hands and eyes can find that displaced wire.
- Judge whether the output is correct — you must test on real hardware.
- Supply your engineering judgment about what the system should actually do.
- Replace the satisfaction of building something real that works.

> ⚠ **IMPORTANT**
>
> AI is your assistant, not your engineer. Copying AI code without understanding it means you cannot debug it when something goes wrong — and something always goes wrong. Read every line, understand every function, and always test the result yourself.

7.3 Project Suggestions

PROJECT A: Smart Lighting System

Automatically control an LED (or lamp) based on ambient light level. The light should turn on when the environment is dark and off when it is bright — mimicking a real streetlight or automatic night light.

PROJECT B: Distance-Based Parking Alert System

Alert a driver (or user) as an object approaches, using increasing buzzer urgency and LED indicators. Reproduce the three-zone logic used in real car parking sensors.

PROJECT C: Intrusion Security Alarm

Detect an intruder entering a monitored zone and trigger an alarm. The system monitors continuously; when any object approaches, it enters alarm mode.

PROJECT D: Automatic Barrier Gate

Detect an approaching vehicle (or object) and automatically open when the object is nearby. Close the gate automatically when the vehicle clears.

7.4 Project Documentation Template

Complete this template before building. An instructor must review your pin table before you receive components.

Field	Your Answer
Project name	
Team members	
Input sensor and type	
Input pin(s) used	
Processing rule (in plain English)	
Output actuator(s)	
Output pin(s) used	
Threshold value and how you will determine it	
Libraries required (if any)	
One challenge you anticipate and how you will address it	
What you would add with more time	

After building:

Field	Your Answer
Actual threshold value used (from calibration)	
Main challenge encountered during building	
How you solved the main challenge	
What worked exactly as expected	

7.5 Assessment Rubric

Projects are evaluated on five criteria. Discuss the rubric with your team before building — it defines "ready for showcase."

Criterion	Outstanding (S)	Good (A)	Satisfactory (B)	Pass (C)	Needs Work (D)
Working system	Fully functional, stable, handles edge cases	Functional with minor glitches	Functional with occasional bugs	Partially functional	Not functional during demo
Sense-Decide-Act explanation	Clear, accurate, confident — all three stages explained	Accurate with minor gaps	Mostly accurate explanation	Partial understanding shown	Unable to explain the loop
Code quality	Well-commented, organised, meaningful variable names	Commented with minor issues	Basic comments present	Sparse comments	No comments; unclear structure
Circuit quality	Neat, correct, colour-coded wires, no loose connections	Correct with minor neatness issues	Mostly correct, some untidiness	Partially correct wiring	Incorrect wiring present
Showcase presentation	Clear, engaging, demonstrates confidently, answers questions	Clear with minor hesitation	Adequate presentation	Minimal presentation given	Not presented

7.6 Showcase Preparation Guide

Your showcase demonstration should take approximately 60–90 seconds. Structure it exactly like this:

1. **State the project name and the problem it solves**
2. **Demonstrate the sense-decide-act loop in action.** Show the trigger happening live.
3. **Describe one challenge you solved.**
4. **State one extension you would build with more time.**

✓ KEY TAKEAWAYS

Every project is the **Input → Processing → Output loop** in a new context. If you can name all three for your project, you understand it well enough to build it.

Build **incrementally** — test the sensor alone, test the actuator alone, then integrate. Never wire the whole thing and hope.

Use the **Serial Monitor** to calibrate thresholds from real data. Never guess a threshold value.

The **Project Documentation Template** forces you to plan before you build — this saves time, not costs it.

Be ready to explain the sense-decide-act loop confidently during the showcase. The demo tells what it does; your explanation tells how and why.

? QUICK REVIEW QUESTIONS

- For each of the four suggested projects, name the specific input sensor, the processing rule in one sentence, and the output actuator.
- Why should you test the sensor reading in Serial Monitor before connecting the actuator?
- How would you determine the correct threshold value for a smart lighting system? Give a specific process.
- Which two modules and which specific components combine to make the automatic barrier gate?
- What is the advantage of writing and testing code in stages rather than writing all at once?
- What is the 60-second showcase structure? Name the four parts.

✏ MINI ACTIVITIES

Design on Paper First: Before building, complete the Project Documentation Template in full. Get your pin table reviewed by your instructor. Only then start wiring.

Threshold Calibration: For any sensor-based project, record 10 Serial Monitor readings under different conditions. Choose your threshold from the data, not from guessing.

Peer Review: After demos, visit another team's project. Using the assessment rubric, give them one specific piece of positive feedback and one specific suggestion for improvement.

Demo Script: Write out your 60-second demo script word for word. Practice it twice. Then put the script away and deliver the demo from memory.

Appendix A
A Gentle Introduction to C Programming
Further Exploration — Optional Reading

Arduino sketches are written in C with a small amount of C++. You can complete the entire workshop using only the code patterns shown in the main modules — but understanding the language itself makes you far more confident when reading tutorials, fixing bugs, and writing your own code from scratch. This appendix is optional enrichment that explains the foundations.

A.1 How Your Code Reaches the Chip

The ATmega328P microcontroller does not understand C. It understands only **machine language** — tiny instructions encoded as binary numbers. Your readable C code must therefore be *translated* before it can run.

There are two translation approaches used in programming:

- **Compiled languages (C, C++):** translated *once*, before running, into a complete machine-code program. Fast to run; memory-efficient; ideal for small, power-limited chips. **Arduino uses this approach.**
- **Interpreted languages (Python, JavaScript):** translated *while running*, one instruction at a time. Easier to write, more flexible, but slower and memory-hungry. Raspberry Pi commonly runs Python.

When you click Upload, the IDE runs this entire pipeline automatically. The single line `a = b + c;` actually generates several machine-code instructions: load the value of `b`, load the value of `c`, add them together, store the result in `a`. The compiler writes these machine instructions for you — this is what compilers are for.

A.2 Memory in the ATmega328P

Memory	Size (Uno)	Volatile?	What It Stores
Flash	32 KB	No — keeps data without power	Your program (sketch)
SRAM	2 KB	Yes — cleared on power-off	Variables, function call stack, during execution
EEPROM	1 KB	No — keeps data without power	User data you explicitly save (using the EEPROM library)

2 KB of SRAM is very small. This is the most common resource that advanced Arduino projects run out of. Symptoms of running out of SRAM include: random resets, unexpected variable values, and Serial output that appears garbled. The solution is to store constant strings in Flash using the `F()` macro: `Serial.println(F("This text stays in Flash"));`

A.3 Variables and Types

A **variable** is a named location in SRAM that holds a value your program can read and modify. In C, every variable has a **type** that you must declare before use. The type tells the compiler how much SRAM to reserve and what operations are valid.

Variable Declaration Examples

```
// Declaration and initialization:
int sensorValue = 0;         // whole number, starts at 0
float voltage   = 3.14;      // decimal number
boolean ledOn   = false;     // true or false only
char letter     = 'A';       // a single character
long bigNumber  = 1000000L; // large whole number; L suffix forces long type

// Const: a variable whose value cannot change after declaration
const int ledPin = 9;        // try to change ledPin later — compiler error

// String of characters (text):
String message = "Hello";    // String type (capital S) — uses dynamic memory
// Prefer: const char* msg = "Hello"; — stores in Flash, saves SRAM
```

Type	Holds	Size (Uno)	Range on Uno	When to Use
boolean	true / false	1 byte	true or false	Flags, on/off states
char	Small integer or character	1 byte	−128 to 127	ASCII characters
int	Whole number	2 bytes (16-bit)	−32,768 to 32,767	Most counters, sensor values
unsigned int	Non-negative whole number	2 bytes	0 to 65,535	When negative is impossible
long	Large whole number	4 bytes (32-bit)	−2.1B to +2.1B	millis(), pulseIn() values
unsigned long	Large non-negative	4 bytes	0 to 4.3B	Always use for millis() variables
float	Decimal number	4 bytes	~6–7 significant digits	Voltage calculations (use sparingly — slow)

A.4 Operators

Arithmetic Operators

Arithmetic Operators

```
int a = 10, b = 3;
```

```
int sum  = a + b;   // 13 — addition
int diff = a - b;   // 7  — subtraction
int prod = a * b;   // 30 — multiplication
int quot = a / b;   // 3  — integer division (remainder discarded)
int rem  = a % b;   // 1  — modulo (remainder of 10 ÷ 3)

a++;   // increment: a becomes 11 (same as a = a + 1)
b--;   // decrement: b becomes 2  (same as b = b - 1)
```

Comparison Operators (used in conditions)

Comparison Operators

```
// All comparison operators return true (1) or false (0)
a == b   // equal to               — TRUE if a equals b
a != b   // not equal to           — TRUE if a does not equal b
a <  b   // less than
a >  b   // greater than
a <= b   // less than or equal to
a >= b   // greater than or equal to

// THE MOST COMMON BEGINNER BUG:
if (a = 5)   // WRONG: assigns 5 to a (always true!)
if (a == 5)  // CORRECT: compares a to 5
```

Logical Operators

Logical Operators

```
// Combine multiple conditions
if (temp > 30 && humidity > 80) { ... }  // AND: both must be true
if (buttonA == LOW || buttonB == LOW) { ... }  // OR: either can be true
if (!buttonPressed) { ... }              // NOT: inverts true/false
```

A.5 Conditionals

Conditionals: if/else and switch

```
// Basic if/else
if (condition) {
  // runs when condition is TRUE
} else {
  // runs when condition is FALSE
}

// Chain with else if
if (light < 200) {
  Serial.println("Dark");
} else if (light < 600) {
  Serial.println("Medium");
} else {
  Serial.println("Bright");
```

```
}

// switch — clean alternative for multiple discrete cases
switch (mode) {
  case 0:
    digitalWrite(greenLED, HIGH);
    break;           // CRITICAL: without break, falls through to next case
  case 1:
    digitalWrite(yellowLED, HIGH);
    break;
  default:           // runs if no case matches
    digitalWrite(redLED, HIGH);
}
```

A.6 Loops

A loop repeats a block of code. Every loop needs: a starting condition, a continue-condition (the loop runs while this is true), and a step that advances toward the end condition.

Loop Types: for, while, do-while

```
// FOR LOOP — use when you know how many iterations
// Structure: for (initialise; condition; step)
for (int i = 0; i < 5; i++) {
  Serial.println(i);  // prints 0, 1, 2, 3, 4
}
// After the loop: i does not exist outside this block

// WHILE LOOP — use when you loop until something changes
while (digitalRead(buttonPin) == HIGH) {
  // waits here until button is pressed (reads LOW with INPUT_PULLUP)
}

// DO-WHILE LOOP — runs at least once, then checks condition
do {
  Serial.println("Checking...");
  reading = analogRead(A0);
} while (reading < 100);

// LOOP CONTROL
break;     // exits the loop immediately
continue;  // skips the rest of the loop body, goes to next iteration
```

A.7 Functions

A **function** groups instructions under a name so you can reuse them. You have been using built-in functions throughout the workshop (`digitalWrite`, `analogRead`, `map`). You can write your own:

Writing Your Own Functions

```
// Function declaration: returnType functionName(parameters) { body }
```

```
// A function that takes two ints and returns their average
int average(int a, int b) {
  return (a + b) / 2;
}

// A function that returns nothing (void) — just does something
void blinkLED(int pin, int times, int duration) {
  for (int i = 0; i < times; i++) {
    digitalWrite(pin, HIGH);
    delay(duration);
    digitalWrite(pin, LOW);
    delay(duration);
  }
}

// Using the functions:
int result = average(10, 20);    // result = 15
blinkLED(13, 3, 200);            // blink pin 13, 3 times, 200ms each

// The getDistance() function in Module 6 is exactly this pattern:
// — groups the 5-line trigger sequence + pulseIn + calculation
// — returns a long (the distance)
// — called with: long dist = getDistance();
```

Good function names describe what the function does (`getDistance`, `openGate`, `soundAlarm`). Functions that are used many times should be functions — it reduces code repetition and makes the main `loop()` much easier to read.

A.8 Common Compiler Error Messages — Plain English

Error Message (partial)	Plain English Meaning	Common Fix
"expected ';' before..."	A semicolon is missing. Usually on the line BEFORE the one flagged.	Add semicolon at the end of the previous line
"was not declared in this scope"	You used a variable or function name before declaring it, or declared it inside a block and tried to use it outside.	Move the declaration to the top of the sketch, before setup()
"no matching function for call to..."	You called a function with wrong argument types or the wrong number of arguments.	Check the function's parameter list and match your call to it
"does not name a type"	You misspelled a type name (e.g., Int instead of int), or used a library type without the #include.	Check spelling; add the required #include at the top
"redefinition of..."	You declared the same variable name twice at the same scope level.	Remove the duplicate declaration; keep only one

"stray '\\xxx' in program"	A non-ASCII character slipped in — often a smart quote (") from copying code from a webpage.	Delete and retype the offending character in the IDE
"ld returned 1 exit status"	The linker failed — usually means a function is called but never defined.	Check that all functions you call are actually written in the sketch
Sketch too big!	Your compiled code exceeds the flash memory (32KB on Uno).	Remove unused libraries; use F() macro for Serial strings; consider a Mega

A.9 The Arduino Standard Functions — Quick Reference

Function	Parameters	Returns	Purpose
pinMode(pin, mode)	pin: 0–13; mode: INPUT/OUTPUT/INPUT_PULLUP	void	Configure a pin as input or output
digitalWrite(pin, val)	pin: 0–13; val: HIGH/LOW	void	Set a digital output pin HIGH (5V) or LOW (0V)
digitalRead(pin)	pin: 0–13	int (HIGH=1 or LOW=0)	Read the state of a digital input pin
analogRead(pin)	pin: A0–A5	int (0–1023)	Read voltage at analog pin; returns 0–1023
analogWrite(pin, val)	pin: PWM-capable; val: 0–255	void	Output PWM signal — dim LEDs, control motors
delay(ms)	ms: milliseconds	void	Pause program for specified milliseconds
delayMicroseconds(us)	us: microseconds	void	Pause for specified microseconds (use for HC-SR04)
millis()	none	unsigned long	Returns milliseconds since board last powered on
Serial.begin(baud)	baud: 9600 (workshop standard)	void	Start serial communication at specified speed
Serial.print(val)	val: any type	void	Print to Serial Monitor — no newline
Serial.println(val)	val: any type	void	Print to Serial Monitor — adds newline at end
tone(pin, freq, dur)	pin; freq in Hz; dur in ms (optional)	void	Generate a square wave at specified frequency
noTone(pin)	pin	void	Stop tone() on the specified pin

pulseIn(pin, state)	pin; state: HIGH or LOW	unsigned long (microseconds)	Measure duration of a pulse on a digital pin
map(val,fL,fH,tL,tH)	value; from and to ranges	long	Rescale a value from one range to another
constrain(val,lo,hi)	value; lower and upper bounds	int/long	Clamp a value to remain within specified bounds
random(min, max)	min (inclusive), max (exclusive)	long	Return a pseudo-random number in the range
randomSeed(seed)	any integer value	void	Seed the random number generator (use analogRead(A0))

Appendix B

Embedded Systems and IoT in Depth

Further Exploration — Optional Reading

B.1 What Makes a System "Embedded"

An **embedded system** is a computer hidden inside another object, dedicated to one task, and designed under tight constraints of cost, power, and size. Three features distinguish embedded design from general programming:

- **Application-specific:** A camera's processor only does camera processing. This focus allows designers to choose exactly the hardware the job requires and nothing more — cheaper and more efficient than a general-purpose computer.
- **Efficiency matters:** It is not enough for code to be correct — it must work within strict limits of memory, power consumption, and response time. A pacemaker must be both reliable *and* run for ten years on a small battery.
- **Hardware and software co-designed:** The programmer must understand the hardware. You cannot write Arduino code without knowing which pins do what, how the ADC works, or how fast the clock runs.

A general-purpose laptop uses a tiny fraction of its power most of the time. An embedded system buys exactly the capability the job requires — which is why the ATmega328P costs a couple of dollars while doing its job perfectly.

B.2 Microcontroller vs Microprocessor

	Microprocessor	Microcontroller
Example	Intel Core i7, Apple M2	ATmega328P (Arduino Uno), ESP32
Speed	GHz range	MHz range (16 MHz for ATmega328P)

Memory	GBs of RAM (external)	KBs of SRAM (on-chip)
Peripherals	Separate chips (GPU, RAM, storage)	Built-in ADC, timers, UART, PWM
Cost	$100–$500+	$1–$10
Power	Watts	Milliwatts
Best for	Operating systems, complex software, video	Dedicated real-time control, sensing, actuating

For tasks that interact with physical sensors and actuators — which operate at human timescales (milliseconds to seconds) — a microcontroller running at 16 MHz is "fast enough." The speed and cost advantages of choosing a microcontroller over a microprocessor are significant in products made in the millions.

B.3 From Embedded to IoT

The Internet of Things is embedded systems plus connectivity. The transition from standalone embedded to IoT became practical because several trends matured simultaneously:

- **Computing got cheaper and smaller:** The cost of a microcontroller capable of Wi-Fi dropped from hundreds of dollars in the 1990s to under $2 today (e.g., the ESP8266).
- **Wireless connectivity became ubiquitous:** Wi-Fi, Bluetooth, cellular networks, and LPWAN protocols now reach most of the world.
- **Cloud platforms commoditised:** Services like AWS IoT, Google Cloud IoT, and Adafruit IO allow developers to connect devices and store data without building server infrastructure.
- **Bandwidth increased:** Real-time sensor data transmission, firmware updates over the air, and video streaming became practical.

A connected device gains abilities a standalone device cannot have: it can offload heavy computation to powerful cloud servers, store and share data online, receive software updates remotely, and be monitored or controlled from anywhere.

B.4 IoT Communication Protocols

As you move beyond the workshop, you will encounter these communication protocols for connecting Arduinos to the internet and to each other:

Protocol	Wires / Range	Speed	Typical Use	Arduino Library
UART / Serial	2 wires (TX/RX), short range	9600–115200 bps	Arduino ↔ computer, Arduino ↔ GPS/GSM/Bluetooth modules	Built-in (Serial)
I2C	2 wires (SDA, SCL), short range	Up to 400 kbps	Multiple sensors on same 2 wires (LCD, IMU, RTC)	Wire.h (built-in)

SPI	4 wires, short range	Up to several MHz	Fast peripherals (SD cards, displays, some sensors)	SPI.h (built-in)
Bluetooth (HC-05)	Wireless, ~10m	~2 Mbps theoretical	Phone-to-Arduino control	SoftwareSerial
Wi-Fi (ESP8266/ESP32)	Wireless, home range	802.11 b/g/n	Internet connectivity, MQTT, HTTP APIs	ESP8266WiFi / WiFi
MQTT (protocol over Wi-Fi)	Wireless, internet-range	Depends on network	IoT data publishing/subscribing to cloud brokers	PubSubClient

B.5 IoT Ethics, Privacy, and Security

The technology you are building has real consequences for real people. These questions are not abstract — they arise in every connected product built today.

Privacy

Every sensor in an IoT device collects data. A fitness band measures your heart rate, location, and sleep. A smart speaker records audio. A connected camera monitors your home. Key questions every designer should ask:

- **What data is being collected?** All of it, or only what is necessary?
- **Where is it stored?** On the device, on a company's servers, in which country, under which laws?
- **Who has access?** The user? The manufacturer? Advertisers? Government authorities?
- **How long is it retained?** Indefinitely? Until you delete your account?
- **Can the user opt out?** If not — why not?

Security

IoT devices with weak security have caused major internet disruptions. A 2016 attack used 100,000 compromised IoT devices (cameras, routers) to overwhelm major websites. Common vulnerabilities:

- **Default passwords:** many devices ship with "admin/admin" or "admin/password" and users never change them. Attackers know this.
- **No update mechanism:** a device that cannot receive firmware updates cannot be patched against newly discovered vulnerabilities. It becomes permanently insecure.
- **Unencrypted communication:** data sent without encryption can be intercepted. Use HTTPS and TLS wherever possible.

Reflection Questions

5. You build a baby monitor using ESP32 with Wi-Fi camera. What are three privacy risks and how would you mitigate each?
6. A company offers a free IoT temperature sensor for your home but charges nothing and ships free. How do they profit? What should you be concerned about?
7. An IoT device you bought five years ago has a security vulnerability. The company has stopped supporting it. What are your options, and what are their tradeoffs?

B.6 Where to Go Next with Arduino

Once you are comfortable with the sensors, actuators, and patterns from this workshop, the natural next steps are:

- **Bluetooth module (HC-05/HC-06):** control your project from a phone app. The HC-05 is a UART-based module — it attaches to the Arduino's serial pins and lets you send and receive text from a paired phone.
- **Wi-Fi capable boards (ESP8266 / ESP32):** the ESP8266 (NodeMCU board) and the ESP32 add Wi-Fi and Bluetooth to an Arduino-compatible platform. They are the gateway to true IoT projects: uploading sensor data to the cloud, receiving commands from a web server, and building web-accessible dashboards.
- **More sensors:** DHT11/DHT22 (temperature + humidity), MQ-2/MQ-135 (gas sensors), PIR (passive infrared motion), soil moisture, rain, ultrasonic — all follow the same Input → Processing → Output pattern you already know.
- **Display modules:** 16×2 LCD (with I2C adapter), OLED displays (SSD1306), and TFT touchscreens give your project a visual interface.
- **Data logging:** write sensor readings to an SD card (SPI-based SD module) for later analysis.
- **Motor drivers (L298N, L293D):** control DC motors bidirectionally to build wheeled robots.

Appendix C — Glossary

ADC (Analog-to-Digital Converter): Hardware inside the microcontroller that converts a continuously varying voltage (0–5V) into a whole number (0–1023) the program can work with.

Actuator: An output device that converts an electrical signal into a physical action. Examples: LED (light), buzzer (sound), servo motor (movement).

analogRead(): Arduino function that reads the voltage on an analog pin (A0–A5) and returns a number from 0 to 1023.

analogWrite(): Arduino function that outputs a PWM signal on a ~ pin (0=off, 255=fully on). Used to dim LEDs and control motors.

Arduino: An open-source electronics platform built around a microcontroller, designed for beginners. The Uno is the standard starter board.

ATmega328P: The main microcontroller chip on the Arduino Uno. Runs at 16 MHz, has 32KB Flash, 2KB SRAM, 1KB EEPROM.

Baud rate: The speed of serial communication in bits per second. Both the Arduino (Serial.begin()) and the Serial Monitor must use the same rate (9600 in this workshop).

Boolean: A data type with only two possible values: true (1) or false (0). Used for flags and binary states.

Breadboard: A reusable plastic board with spring-connected holes used to build circuits without soldering. Rows of 5 holes are connected; side rails carry power and ground.

Buzzer (active): An output device that produces a tone when powered. An active buzzer has an internal oscillator and beeps with digitalWrite(HIGH). A passive buzzer requires tone().

Compiler: Software that translates human-readable C code into binary machine code the microcontroller can execute.

const: C keyword declaring a variable whose value cannot change after initialisation. Use for pin numbers and fixed thresholds.

constrain(): Arduino function that clamps a value to stay within a specified range. constrain(val, 0, 255) ensures val stays between 0 and 255.

delay(): Arduino function that pauses the program for a specified number of milliseconds. Nothing else happens while the program is paused.

delayMicroseconds(): Like delay() but accepts microseconds (millionths of a second). Required for generating the HC-SR04 trigger pulse.

digitalRead(): Arduino function that reads the state of a digital input pin. Returns HIGH (1) or LOW (0).

digitalWrite(): Arduino function that sets a digital output pin to HIGH (5V) or LOW (0V).

Duty cycle: In PWM, the percentage of each period the signal spends HIGH. 50% duty cycle = equal on and off times = half average power.

ECHO (HC-SR04 pin): The HC-SR04 pin that goes HIGH for the duration of the ultrasonic round-trip travel time. Read with pulseIn().

Echolocation: Measuring distance by emitting a sound pulse and timing how long the echo takes to return. Used by bats, dolphins, and the HC-SR04.

Embedded system: A computer hidden inside another object, dedicated to a specific task. Examples: microwave, anti-lock braking system, digital watch.

Flash memory: Non-volatile memory inside the ATmega328P that stores your program. Contents survive power-off. 32KB on the Arduino Uno.

Floating pin: A digital input pin with no defined connection to either HIGH or LOW. It picks up electromagnetic noise and reads randomly. Fixed by using INPUT_PULLUP or an external resistor.

for loop: A programming construct that repeats a block of code a fixed number of times. Structure: for (initialise; condition; step) { body }.

GND: Ground — the 0V reference point for all circuits. Every component's negative connection must eventually reach a GND pin.

HC-SR04: The ultrasonic distance sensor used in this workshop. Measures distance from 2cm to 400cm using echolocation. Four pins: VCC, GND, TRIG, ECHO.

HIGH: In Arduino, the state representing approximately 5V on a digital pin. Equivalent to 1, ON, or true.

IDE (Integrated Development Environment): The software application used to write, verify, and upload Arduino code. Bundles text editor, compiler, uploader, and Serial Monitor.

INPUT_PULLUP: A pinMode mode that connects an internal 10kΩ pull-up resistor to a pin. Default state is HIGH; pressing a button to GND reads LOW. The correct way to wire a button without an external resistor.

IoT (Internet of Things): Embedded systems connected to the internet, enabling remote monitoring, control, and data sharing.

LDR (Light Dependent Resistor): A sensor whose resistance decreases in bright light and increases in darkness. Used with a voltage divider to produce a variable voltage for analogRead().

LED (Light Emitting Diode): A polarised semiconductor device that emits light when current flows in the correct direction. Long leg = anode (+), short leg = cathode (−).

Library: A collection of pre-written, tested functions that you include in your sketch with #include. Examples: Servo.h, Wire.h, EEPROM.h.

loop(): The second essential Arduino function. Runs continuously after setup() completes, repeating from top to bottom indefinitely.

LOW: In Arduino, the state representing 0V on a digital pin. Equivalent to 0, OFF, or false.

map(): Arduino function that proportionally rescales a value from one numeric range to another. map(val, 0, 1023, 0, 255) converts ADC range to PWM range.

millis(): Arduino function returning the number of milliseconds since the board last powered on. Returns unsigned long. Used for non-blocking timing.

Microcontroller: A complete computer on a single chip: processor, memory, and programmable I/O. Designed for embedded control tasks. The Arduino Uno uses the ATmega328P.

Microprocessor: A powerful, general-purpose CPU chip found in computers and phones. More powerful than a microcontroller but requires external RAM, storage, and peripherals.

pinMode(): Arduino function that configures a digital pin as INPUT, OUTPUT, or INPUT_PULLUP. Must be called in setup() before using the pin.

Potentiometer: A three-pin variable resistor with a rotating shaft. Wired as a voltage divider: outer pins to 5V and GND, middle pin (wiper) to analog input.

Pull-up resistor: A resistor connected between a pin and 5V that holds the pin HIGH when nothing else is driving it. Prevents floating. The Arduino's internal pull-up (~10kΩ) is enabled with INPUT_PULLUP.

pulseIn(): Arduino function that measures the duration (in microseconds) that a specified pin stays in a specified state (HIGH or LOW). Used to read the HC-SR04 ECHO pulse.

PWM (Pulse Width Modulation): A technique for producing variable average power by rapidly switching a digital output ON and OFF. Duty cycle controls the average. Available on ~ pins on the Arduino Uno.

random(): Arduino function returning a pseudo-random number between specified minimum and maximum values.

Resistor: A component that limits current flow. Measured in ohms (Ω). Used to protect LEDs (220Ω) and to create pull-up/pull-down circuits (10kΩ).

Sensor: An input device that detects a physical quantity (light, distance, temperature, pressure) and converts it to an electrical signal.

Serial Monitor: A window in the Arduino IDE that shows text sent from the Arduino over USB. Used for debugging and live sensor monitoring.

Servo motor: A motor with built-in position feedback that moves to a precise, held angle (typically 0°–180°). Controlled with the Servo library using write(angle).

setup(): The first essential Arduino function. Runs exactly once when the board powers on or resets. Used for pin configuration and initialisation.

Sketch: The Arduino term for a program. A sketch always contains setup() and loop().

SRAM: Static RAM — volatile memory inside the ATmega328P that stores variables during program execution. 2KB on the Arduino Uno. Contents are lost when power is removed.

tone(): Arduino function that generates a square wave at a specified frequency on a digital pin. Used to produce musical tones from a passive buzzer.

TRIG (HC-SR04 pin): The HC-SR04 pin that receives a 10µs HIGH pulse from the Arduino to start a distance measurement.

Upload: The process of compiling and transferring a sketch from the Arduino IDE to the Arduino board's flash memory via USB.

Voltage divider: Two resistors in series between 5V and GND. The voltage at the junction between them is proportional to the ratio of the two resistors. Required for reading LDR sensors.

Appendix D — Component Reference Card

Quick-reference specifications for every component in the workshop kit.

LED (Light Emitting Diode)

Specification	Value
Forward voltage (Vf)	~2.0V (red/yellow/green); ~3.0V (blue/white)
Maximum forward current	20mA (do not exceed)
Safe working current	10–15mA with 220Ω resistor from 5V
Polarity	Anode (+) = long leg; Cathode (−) = short leg
Resistor needed?	YES — always use 220Ω or 330Ω from 5V
Colour-coded resistor (220Ω)	Red – Red – Brown – Gold

Push Button (Tactile Switch)

Specification	Value
Type	Momentary SPST (normally open)
Legs	4 legs in two internally-connected pairs; straddle centre channel of breadboard
wiring	One leg to digital pin; other leg to GND; use INPUT_PULLUP — no external resistor needed
Reading when pressed	LOW (with INPUT_PULLUP)
Reading when released	HIGH (with INPUT_PULLUP)
Debounce	Add delay(50) after detecting press for reliable operation

Potentiometer (10 kΩ)

Specification	Value
Type	Rotary, 270° range
Total resistance	10 kΩ (pin 1 to pin 3)
Pin 1 (left)	Connect to 5V
Pin 2 (middle — wiper)	Connect to analog input (A0)
Pin 3 (right)	Connect to GND
Output range	0V to 5V at wiper → analogRead() returns 0–1023
External resistor needed?	NO — self-contained voltage divider

LDR (Light Dependent Resistor / Photoresistor)

Specification	Value
Resistance in bright light	~200Ω – 1kΩ (varies by type)
Resistance in darkness	~20kΩ – 1MΩ (varies by type)
Polarity	None — connect either way
Wiring	LDR from 5V to A0; 10kΩ resistor from A0 to GND (voltage divider — MANDATORY)
Output in bright light	HIGH analogRead value (~800–1023)
Output in darkness	LOW analogRead value (~0–200)

Active Buzzer

Specification	Value
Type	Active (internal oscillator) — produces fixed tone with just 5V
Operating voltage	3.5V – 5.5V
Current draw	~30mA
Polarity	+ marking on top. Connect + to digital pin; other leg to GND
Control method	digitalWrite(pin, HIGH) = beep; LOW = silent
Tone control	Use tone() for specific frequencies (if using passive buzzer)

SG90 Micro Servo Motor

Specification	Value
Angular range	0° to 180° (approximately)
Stall torque	1.8 kg/cm at 5V
No-load current	~150mA
Stall current	Up to 750mA
Control signal	Orange wire → any digital pin (suggest pin 6 or 9)
Power (red wire)	Connect to Arduino 5V (single servo) or external 5V (multiple)
Ground (brown wire)	Connect to GND
Library	#include <Servo.h> — call attach(pin) then write(angle)

HC-SR04 Ultrasonic Sensor

Specification	Value
Operating voltage	5V DC
Current draw	~15mA during measurement
Measuring range	2cm – 400cm (reliable up to ~200cm)
Accuracy	±3mm under ideal conditions
Detection angle	~15° cone
Frequency	40 kHz (ultrasonic)
VCC pin	Connect to Arduino 5V
GND pin	Connect to GND
TRIG pin	Connect to digital OUTPUT pin; send 10µs HIGH pulse to start measurement
ECHO pin	Connect to digital INPUT pin; reads HIGH for duration of round-trip time
Distance formula	distance_cm = pulseIn(echoPin, HIGH) × 0.034 / 2
Data type for duration	Use long (not int) — avoids integer overflow

Appendix E — Arduino Programming Quick Reference

A one-page condensed reference for all functions and patterns used in this course.

Essential Sketch Structure

```
#include <LibraryName.h>          // if using a library

const int PIN_NAME = 9;           // pin numbers at top — easy to change
int variableName  = 0;            // working variables also at top

void setup() {                    // RUNS ONCE at power-on
  pinMode(PIN_NAME, OUTPUT);      // configure pin direction
  Serial.begin(9600);             // start Serial Monitor
}

void loop() {                     // RUNS FOREVER
  // your code here
}
```

Digital I/O

```
pinMode(pin, OUTPUT);             // configure pin as output
pinMode(pin, INPUT);              // configure pin as input (floating — avoid)
pinMode(pin, INPUT_PULLUP);       // input with internal pull-up (PREFERRED for buttons)

digitalWrite(pin, HIGH);          // set output HIGH (5V)
digitalWrite(pin, LOW);           // set output LOW (0V)

int state = digitalRead(pin);     // read input: returns HIGH(1) or LOW(0)
```

Analog I/O

```
int raw = analogRead(A0);         // read analog pin: 0-1023 (A0-A5 only)

analogWrite(pin, value);          // PWM output: 0-255 (~pins only: 3,5,6,9,10,11)
```

Timing

```
delay(1000);                      // pause 1000 ms (blocking)
delayMicroseconds(10);            // pause 10 µs (blocking, short pauses)

unsigned long t = millis();       // ms since power-on (non-blocking stopwatch)
// Non-blocking pattern:
if (millis() - lastTime >= 500) { lastTime = millis(); /* action */ }
```

Serial Monitor

```
Serial.begin(9600);                 // start serial (in setup())
Serial.print("Label: ");             // print without newline
Serial.println(value);              // print with newline
Serial.print("x="); Serial.println(x);  // label + value on one line
```

Math and Logic

```
int b = map(a, 0, 1023, 0, 255); // rescale from one range to another
int c = constrain(b, 0, 255);    // clamp value to range
long r = random(100, 500);       // random number 100–499
randomSeed(analogRead(A0));      // seed the random generator (in setup())
```

Servo Motor

```
#include <Servo.h>
Servo myServo;
myServo.attach(9);                  // link object to signal pin
myServo.write(angle);               // move to angle (0–180)
myServo.detach();                   // stop generating pulses (servo relaxes)
```

HC-SR04 Distance Sensor

```
// Trigger sequence (TRIG = output pin):
digitalWrite(trigPin, LOW);  delayMicroseconds(2);
digitalWrite(trigPin, HIGH); delayMicroseconds(10);
digitalWrite(trigPin, LOW);
// Read echo (ECHO = input pin):
long duration = pulseIn(echoPin, HIGH);  // returns µs as long
long distance  = duration * 0.034 / 2;   // convert to cm
if (duration == 0) distance = 999;       // out-of-range guard
```

Sound

```
tone(pin, frequency);           // start tone at frequency Hz (passive buzzer)
tone(pin, frequency, duration); // tone for duration ms then stops
noTone(pin);                    // stop the tone
digitalWrite(pin, HIGH);        // active buzzer on
digitalWrite(pin, LOW);         // active buzzer off
```

Appendix F — Comprehensive Troubleshooting Guide

Consult this guide when something is not working. Work through each section in order.

Step 1: Board and Connection

Symptom	Likely Cause	Fix
Power LED not on	No power; USB cable not connected or charge-only	Try a different cable; try a different USB port on computer
No ports appear in IDE	Missing USB driver (Windows) or charge-only cable	Reinstall drivers; swap cable; check Device Manager on Windows
"Programmer not responding"	Wrong board selected	Tools → Board → Arduino AVR Boards → Arduino Uno
"Port not found"	Wrong port or board disconnected	Check Tools → Port; unplug and replug board
Board disconnects during upload	USB power issue or boot-loader corruption	Try different USB port; try direct connection (no hub)

Step 2: Compilation Errors

Error Message	Cause	Fix
"expected ';' before..."	Missing semicolon (usually on PREVIOUS line)	Add ; at end of line before the highlighted one
"was not declared in this scope"	Variable declared inside a block; used outside it	Move declaration to top of sketch, before setup()
"no matching function"	Wrong function name or wrong argument types/count	Check exact function name and parameter list in Appendix E
"does not name a type"	Misspelled type name or missing #include	Check spelling; add #include <LibraryName.h>
"redefinition of..."	Same variable name declared twice	Remove the duplicate; keep only one declaration
"Sketch too big"	Code exceeds 32KB flash	Remove unused libraries; use F() for Serial strings

Step 3: LED Problems

Symptom	Cause	Fix
LED does not light at all	Reversed polarity (anode/cathode swapped)	Flip the LED — long leg toward resistor/signal
LED does not light	No resistor; missing connection; wrong pin in code	Check: 220Ω resistor present? Code pin matches wiring?
LED is very dim	High-value resistor; underpowered PWM signal	Use 220Ω; verify analogWrite value is not near 0
LED burned out (flash then dead)	Connected directly to 5V with no resistor	Replace LED; always wire resistor first
LED always on / always off with button	Using INPUT mode (floating); or = instead of == in if	Use INPUT_PULLUP; check condition for == not =
analogWrite dimming not working	LED on a non-PWM pin	Move LED to pin 3, 5, 6, 9, 10, or 11 (~)

Step 4: Button Problems

Symptom	Cause	Fix
Button always reads HIGH (not pressed)	Floating pin — using INPUT without pull-up	Change to INPUT_PULLUP; no external resistor needed
Button reads 0 and 1 rapidly while held	Mechanical bounce	Add delay(50) after detecting press
Button reads wrong value	Using = not == in condition	Change to == for comparison
Button acts as if always pressed	Button across wrong breadboard rows (both pins same side of channel)	Straddle button across the centre channel of breadboard
Button pressed but nothing happens	Wrong pin in code; loose wire	Add Serial.println(digitalRead(pin)) to verify reading

Step 5: Analog Sensor Problems

Symptom	Cause	Fix
LDR reads same in all light conditions	Missing 10kΩ voltage divider resistor	Add 10kΩ from A0 to GND; LDR from 5V to A0
Analog reading always 0 or 1023	Wrong pin; wiring issue	Verify connection to A0–A5; check power connections
Readings jump ±10 randomly	Normal ADC noise	Average 5 readings; use constrain() after map()

Serial shows garbled text	Baud rate mismatch	Match Serial.begin() value to Serial Monitor dropdown
map() gives unexpected value	Integer truncation; input outside declared fromLow–fromHigh	Add constrain() before and after map()

Step 6: Servo Problems

Symptom	Cause	Fix
Servo jitters at rest	Power supply insufficient or noisy	Power servo from external 5V; add capacitor across servo power
Servo does not reach 0° or 180°	Mechanical end-stop; SG90 actual range may be 170°	Use constrain(angle, 5, 175); find physical limits
Servo buzzes under load	Load too heavy (stall); SG90 stall torque is 1.8kg/cm	Reduce load; use larger servo
Servo twitches when write() called rapidly	Too many write() calls per second	Add delay(15) between writes
"Servo.h" error	Library missing (rare — usually pre-installed)	Tools → Manage Libraries → search "Servo" → install

Step 7: HC-SR04 Problems

Symptom	Cause	Fix
Always reads 0	No echo returned; TRIG or ECHO wiring issue	Verify TRIG→OUTPUT, ECHO→INPUT; confirm object in range
Reads huge numbers (millions)	Integer overflow — int used instead of long	Change int duration, distance to long
Readings jump ±20cm	Nearby hard reflective surface; interference	Move sensor away from walls; ensure target faces sensor squarely
Trigger pulse not working	Using delay(10) instead of delayMicroseconds(10)	Use delayMicroseconds(10) for the 10µs trigger pulse
Close objects read 0	Object closer than 2cm minimum range	Move object further away; check for duration == 0

Step 8: Serial Monitor Problems

Symptom	Cause	Fix
Nothing appears in monitor	Serial.begin() missing from setup()	Add Serial.begin(9600) as first line of setup()
Garbled / random characters	Baud rate mismatch	Set both Serial.begin() and monitor dropdown to 9600
Values do not update	Missing delay() or Serial.print() in loop()	Add Serial.println(value) and small delay(100) in loop()
Print shows wrong value	Reading variable before it is updated	Verify variable is updated before Serial.print() call

The Golden Debugging Sequence

When nothing is working and you do not know where to start:

8. **Unplug the USB.** This resets both the board and prevents further damage if something is shorted.
9. **Check power first.** Is 5V present at the power rail? Is GND connected?
10. **Verify each connection.** Trace every wire from its source to its destination. One row off on the breadboard is the most common error.
11. **Isolate the problem.** Remove all but the simplest circuit (a single LED with Blink). Does that work? If yes, add one component at a time.
12. **Use Serial Monitor to see inside.** Add `Serial.println()` before and after every suspicious line of code.
13. **Check the code for = vs ==.** Search for every `if` statement and verify == is used for comparison.
14. **For buttons: add INPUT_PULLUP.** Change every `INPUT` to `INPUT_PULLUP` for buttons. Update the condition logic (LOW = pressed).
15. **For sensors: print the raw value first.** Before any threshold logic, print the analogRead() value and verify it makes physical sense.
16. **Fix one error at a time.** Compile, fix the first error, compile again. Multiple errors are often caused by one root mistake.
17. **Ask a partner.** A fresh pair of eyes finds the misplaced wire in seconds.

References and Further Learning

Workshop Resources

- **TinkerCad Circuits** — Free online Arduino simulator. No hardware needed to test circuits and code. Ideal for pre-workshop preparation and experimentation at home. https://www.tinkercad.com/circuits
- **Arduino Project Hub** — Official repository of hundreds of community-contributed tutorials, project walkthroughs, and code examples. https://projecthub.arduino.cc/

- **Arduino Language Reference** — Complete documentation for every built-in function, type, and structure in the Arduino language. The definitive reference. https://www.arduino.cc/reference/en/
- **Arduino Official Documentation** — Board pinouts, IDE guides, getting started tutorials. https://docs.arduino.cc/
- **Instructables — Arduino Projects** — Step-by-step project tutorials for all skill levels, with photographs and code. https://www.instructables.com/circuits/arduino/projects/

Recommended Books

- **Programming Arduino: Getting Started with Sketches** by Simon Monk — The friendliest first book for absolute beginners. Covers exactly the material in this workshop and a step beyond. Widely available.
- **Make: Electronics** by Charles Platt — A hands-on, experiment-driven introduction to electronics fundamentals. Learn Ohm's Law, capacitors, transistors, and ICs by actually burning things out (safely). Excellent foundation for hardware understanding.
- **Exploring Arduino** by Jeremy Blum — Bridges beginner Arduino projects toward intermediate embedded concepts. Excellent for self-study after this workshop.
- **Arduino Cookbook** by Michael Margolis and Brian Jepson — A recipe-format reference (over 200 worked examples) covering common tasks from blinking LEDs to networking.

Online Courses

- **Coursera: Introduction to the Internet of Things and Embedded Systems** (University of California, Irvine) — A gentle MOOC covering IoT concepts, embedded systems theory, and introductory programming. The conceptual material aligns directly with Appendix B of this handbook. Free to audit.
- **Coursera Specialisation: An Introduction to Programming the Internet of Things (IoT)** — Five-course sequence using Arduino and Raspberry Pi, from the same institution. Recommended next step after this workshop.
- **Random Nerd Tutorials** — Excellent practical tutorials for ESP8266, ESP32, and IoT projects using the Arduino IDE. Free. https://randomnerdtutorials.com/

Open Source Project Ideas — Progressively Challenging

Beginner (Modules 1–4 skills)

- Traffic light sequencer with three LEDs
- Knob-controlled LED dimmer
- Automatic night light using an LDR
- Reflex reaction-time game with leaderboard
- Morse code message blinker

Intermediate (Adds Modules 5–6)

- Potentiometer-controlled servo arm with Serial readout
- Ultrasonic distance display on Serial Monitor with buzzer alert
- Smart dustbin (ultrasonic sensor + servo lid)
- Automatic barrier gate (ultrasonic + servo)
- Colour-mixing LED (three PWM-controlled LEDs, RGB mixing)

Advanced (Beyond this workshop)

- Bluetooth-controlled robot car — HC-05 module + L298N motor driver + DC motors + phone app
- Weather station — DHT11/DHT22 temperature+humidity + barometric pressure sensor + Serial data log
- Wi-Fi temperature logger — ESP32 + DHT22 + MQTT broker + web dashboard
- Plant-watering system — soil moisture sensor + threshold detection + relay to control a pump
- Line-following robot — IR sensors + motor driver + PID control algorithm